République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieure et de la Recherche Scientifique



جامعة مولود معمري تيزي وزو
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵎⵓⵍⵓⴷ ⵎⵄⵎⵔⵉ ⴰⵜ ⵣⵉⵣⵓ
UNIVERSITÉ MOULOUD MAMMERI DE TIZI-OUZOU



Faculté du Génie Electrique et d'Informatique

Département d'Informatique

MEMOIRE DE FIN DE CYCLE

En vue de l'obtention du diplôme de Licence en Informatique

Option : Système d'information

THEME : Conception et réalisation d'une application web de gestion intégrée pour auto-école

Réalisé par :

- BOUSSAD Manal
- HAMMI Hanane
- LOUNI Elena

Dirigé par :

- **Mr. Ounnaci**

Promotion : 2024/2025

Remerciement

Nous tenons tout d'abord à exprimer notre profonde gratitude à nos parents et à notre famille pour leur soutien inconditionnel, leurs sacrifices et leurs encouragements constants tout au long de notre parcours universitaire. Leur amour et leur confiance ont été notre plus grande force.

Nous souhaitons adresser nos remerciements les plus sincères à Monsieur Ounnaci, notre encadrant, pour son expertise, sa disponibilité et ses précieux conseils qui ont guidé chacune des étapes de ce travail. Son professionnalisme et sa rigueur scientifique ont été déterminants dans la réalisation de ce mémoire.

Nous tenons également à remercier tous nos professeurs du département d'informatique pour leurs enseignements de qualité et leur transmission de savoir, qui ont forgé notre formation académique. Un merci particulier à nos ami(e)s et collègues pour l'ambiance chaleureuse et l'entraide fraternelle qui ont rendu ce parcours si enrichissant.

Enfin, nous n'oublions pas toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de ce travail. Que chacun trouve ici l'expression de notre sincère reconnaissance.

Dédicace :

Ce travail est dédié :
À nos parents, pour leur inestimable soutien et leurs sacrifices,
À nos famille et nos ami(e)s, pour leur encouragement constant,
Et à tous ceux qui, par leur affection ou leur exemple, ont contribué à sa réalisation.

Résumé :

Dans le cadre de ce projet de fin de cycle, nous avons développé une application web intégrée pour la gestion des auto-écoles. Cette solution complète permet aux gérants d'administrer efficacement leur établissement. Les candidats bénéficient d'un espace personnel en ligne pour suivre leur formation. Les moniteurs disposent quant à eux d'une interface dédiée à leurs activités. Cette application centralisée offre une digitalisation complète des processus métiers, améliorant ainsi la productivité de l'auto-école tout en offrant une meilleure expérience utilisateur.

Abstract :

As part of this final year project, we developed an integrated web application for driving school management. This comprehensive solution enables administrators to efficiently manage their institution. Students benefit from a personal online space to monitor their training progress. Instructors are provided with a dedicated interface for their activities. This centralized application offers complete digitalization of business processes, thereby enhancing the driving school's productivity while delivering an improved user experience.

Sommaire :

Introduction générale ……..... 7

Contexte et justification ……... 7

Problématique …… 7

Objectifs (généraux et spécifique)……..... 7

Méthodologie adoptée ……. 8

Organisation du document ……... 8

Chapitre 1 : Etude de l'existant ……9

1. Fonctionnement et Cadre Réglementaire d'une Auto-École Classique en Algérie .10

2 .Le fonctionnement d'une auto-école classique ……11

3. Limites et Contraintes des Systèmes Actuels : Analyse SWOT ……13

Chapitre2 : Analyse des besoins ……..……15

1. Recueil des besoins fonctionnels et non fonctionnels ……..……15

1.1 Besoins fonctionnels……16

1.2 Besoins non fonctionnels …….…....17

2. Acteur du système…….20

3. Les cas d'utilisations principaux…… 20

4 . Les diagrammes UML…… 22

4.1 Diagramme de contexte ……23

4.2 Diagramme des cas d'utilisation ……24

4.3 Diagramme de séquences ……. 25

4.4 Diagramme de classes …….. 31

4.5 Diagramme de déploiement ……. 32

Chapitre 3 : Conception de la solution…….……33

1. Les choix technologiques ……34

2 Architecture Client-serveur ……36

Protocole de communication ……37

3.Conception de la base de données ……..37

Modèle logique de données .....….40

4. maquette de l'interface utilisateur (authentification)…… 41

Chapitre 4 :Développement de la solution …… 44

Structure du projet…… 45

Présentation de l'Interface Utilisateur et Fonctionnalités de l'Application …….48

Conclusion et perspectives ……..62

Bibliographie……...……63

Table des figures :

Figure 01 : « Diagramme de Contexte»

Figure 02 : «Diagramme de Cas d'utilisation : Acteur Administrateur »

Figure 03 : «Diagramme de Cas d'utilisation : Acteur Moniteur »

Figure 04 : «Diagramme de Cas d'utilisation : Acteur Candidat »

Figure 05 : «Diagramme de séquence-Authentification- »

Figure 06 : «Diagramme de séquence-Ajout d'un candidat- »

Figure 07 : «Diagramme de séquence-Recherche- »

Figure 08 : «Diagramme de séquence-Suppression d'un candidat- »

Figure 09: «Diagramme de séquence-Modification d'un candidat- »

Figure 10: «Diagramme de séquence-Gérer les comptes utilisateurs »

Figure 11: «Diagramme de séquence-Planifier séances »

Figure 12: «Diagramme de séquence-Planifier examen/Résultat examens »

Figure 13: «Diagramme de séquence-Consulter fiches personnelles moniteurs/ candidat / plannings examens / séances »-admin-

Figure 14: «Diagramme de séquence-Consulter fiches personnelles candidat / plannings examens / séances /Résultats »-moniteurs-

Figure 15: «Diagramme de séquence-Consulter fiches personnelles / plannings examens / séances /Résultats »-candidats-

Figure 16 : «Diagramme de classes »

Figure 17 : «Diagramme de déploiement »

Figure 18 : «Maquette authentification »

Introduction générale :

Contexte et justification :

Dans un environnement de plus en plus digitalisé, la modernisation des systèmes de gestion devient essentielle pour toutes les organisations, y compris les auto-écoles. Les technologies de l'information offrent désormais des solutions indispensables pour fluidifier les échanges, réduire les contraintes de temps et d'espace, et optimiser les coûts. Conscientes de ces enjeux, les auto-écoles manifestent un intérêt croissant pour les outils de gestion numériques.

Notre projet propose précisément le développement d'une application web spécialisée pour ce secteur. Cette solution vise à digitaliser les processus quotidiens, moderniser la gestion administrative et pédagogique, et renforcer l'efficacité organisationnelle. Elle répond ainsi aux besoins de transformation digitale des auto-écoles en offrant une gestion plus performante et adaptée aux réalités du terrain.

Problématique :

Les auto-écoles souffrent encore d'une gestion trop artisanale. L'usage excessif du papier (dossiers d'inscription, registres de suivi) rend les informations difficiles à retrouver et favorise les erreurs. La planification des cours repose sur des échanges verbaux, ce qui devient ingérable dès qu'un moniteur est absent ou qu'une leçon doit être reportée - obligeant le personnel à une fastidieuse campagne d'appels téléphoniques. Les élèves et moniteurs manquent cruellement d'autonomie, devant systématiquement contacter le secrétariat pour la moindre information.

Cette organisation désuète engendre trois problèmes majeurs : une charge administrative accablante pour le personnel, une frustration croissante des usagers, et des pertes financières liées à cette inefficacité. Dans un secteur où la concurrence se digitalise, ces établissements risquent de prendre du retard. Notre solution web répond précisément à ces enjeux en automatisant les processus clés et en offrant à chacun un accès instantané aux informations qui le concernent.

Objectifs du projet :

1. Objectif général :

Développer une application web intégrée pour la gestion numérique complète des auto-écoles, offrant un accès personnalisé et authentifié aux différents acteurs (gestionnaires, moniteurs, candidats), permettant de moderniser et d'optimiser l'ensemble des processus administratifs et pédagogiques.

2. Objectifs spécifiques :

2.1. Pour la gestion administrative :

- Digitaliser le suivi des inscriptions et dossiers des candidats

- Automatiser la gestion des plannings et emplois du temps
- Centraliser la gestion des ressources (moniteurs, véhicules)
- Sécuriser l'archivage des données
- Gestion des comptes utilisateurs

2.2. Pour les moniteurs (après authentification)

- Consultation des résultats d'examen
- Accès aux fiches individuelles des candidats
- Visualisation de leur fiche personnelle
- Consultation du planning des examens théoriques et pratiques
- Consultation du planning des séances.

2.3. Pour les candidats (après authentification)

- Consultation des résultats d'examen
- Accès à leur fiche personnelle
- Visualisation du planning des examens et des séances
- Ressources pédagogiques en ligne :

2.3 Sur le plan technique, l'objectif est de proposer une interface simple, de garantir la sécurité des données personnelles et de prévoir une architecture évolutive pour intégrer facilement de nouvelles fonctionnalités.

La méthodologie adoptée :

Dans ce projet, nous avons adopté une méthodologie structurée combinant des outils de modélisation, de développement et d'analyse. Pour comprendre le fonctionnement et la structure d'une auto-école, nous avons eu recours à des techniques documentaires (livres, sites web), des observations sur le terrain ainsi que des entretiens avec le personnel. La modélisation UML a été réalisée avec l'outil Draw.io, notamment pour concevoir les cas d'utilisation. Le développement de l'application a été effectué en PHP, un langage couramment utilisé pour le développement web. L'interface utilisateur a été conçue avec les technologies HTML, CSS et JavaScript. La base de données a été gérée via MySQL en utilisant la plateforme XAMPP. L'ensemble du développement a été réalisé dans l'environnement Visual Studio Code, assurant ainsi une cohérence entre la conception et l'implémentation.

L'organisation du document :

Ce mémoire est structuré en quatre chapitres. Le premier traite de l'étude de l'existant. Le second présente l'analyse des besoins et les diagrammes UML. Le troisième est consacré à la conception de la solution (choix technologiques, architecture, base de données). Enfin, le quatrième chapitre décrit le développement et la mise en œuvre de l'application, suivi d'une conclusion générale.

CHAPITRE 1 :

ETUDE DE L'EXISTANT

L'étude de l'existant est une étape essentielle qui permet de comprendre le fonctionnement actuel du système, d'identifier ses limites et de cerner les besoins des utilisateurs. Dans ce chapitre, nous analysons le mode de gestion traditionnel d'une auto-école, encore souvent basé sur des outils manuels comme les registres papier ou les tableurs non intégrés. Cette analyse met en évidence les faiblesses du système actuel et souligne l'intérêt d'une solution numérique mieux adaptée aux réalités du terrain.

1. Fonctionnement et Cadre Réglementaire d'une Auto-École Classique en Algérie :

1.1Conditions et modalités d'ouverture des auto-écoles :

L'ouverture d'une auto-école en Algérie nécessite un agrément délivré par le wali, sur avis d'une commission compétente .Pour les *personnes physiques*, plusieurs conditions sont requises : avoir au moins 21 ans, être de nationalité algérienne, jouir de ses droits civiques, présenter des garanties de moralité, justifier d'une capacité professionnelle (diplôme et Certificat d'Aptitude Pédagogique et Professionnelle ((CAPP)), d'une assurance et de garanties financières. Pour les *personnes morales*, les dirigeants doivent répondre aux mêmes critères, et l'établissement doit disposer d'un local conforme, de matériel pédagogique adapté, et de véhicules équipés pour l'enseignement.

Selon la réglementation, toute personne enseignant la conduite à titre onéreux doit être titulaire du CAPP, délivré après une formation spécialisée et un examen.

1.2Le Centre National du Permis de Conduire (CENAPEC) :

Le CENAPEC, placé sous la tutelle du ministère des Transports, encadre les activités de formation à la conduite. Il met en œuvre une politique nationale visant à améliorer la qualité de l'enseignement à travers des outils pédagogiques modernes. Il supervise aussi les examens, élabore une carte nationale des épreuves et peut effectuer des contrôles sur les moniteurs et examinateurs.

1.3 Le permis de conduire

Le permis de conduire est un document officiel utilisé pour la conduite selon des catégories de véhicules. Son obtention, son renouvellement ou sa conversion nécessitent un dossier administratif à déposer auprès des autorités locales (daïra, wilaya). Tout citoyen remplissant les conditions légales peut y accéder via une auto-école agréée.

1.3.1 La procédure d'obtention :

Selon le décret exécutif n°04-381, un permis est requis pour conduire. Il est délivré après avis favorable d'un examinateur, sous conditions d'âge :

- 16 ans pour A1 et F
- 18 ans pour A2, B et F

- 25 ans pour C1, C2, D et E

1.3.2 Durée de validité :

- 2 ans : catégorie F
- 5 ans : C1, C2, D, E
- 10 ans : A1, A2, B

1.3.3 Catégories du permis :

- **A1 :** permet de conduire des motos légères jusqu'à 125 cm³
- **A2** : autorise la conduite de motos intermédiaires (jusqu'à 35 kW)
- **B** : véhicules ≤ 3 500 kg, max 9 passagers
- **C1 :** pour les petits camions de transport de marchandises entre 3,5 et 7,5 tonnes.
- **C2** : permet de conduire des camions de plus de 7,5 tonnes
- **D** : transport de plus de 8 personnes
- **E** : véhicules avec remorque lourde
- **F** : véhicules adaptés pour personnes en situation de handicap

1.4 Examen du permis :

Composé d'une épreuve théorique (code) et de deux épreuves pratiques :

- Manœuvres sur terrain (Créneaux / garage)
- Conduite en circulation

1.5 Dossier d'examen :

Le candidat (≥19 ans ou autorisation parentale) doit fournir :

- Formulaire légal, acte de naissance, certificat de résidence
- Carte de groupe sanguin, certificat médical
- 6 photos, timbre fiscal
- Paiement des frais d'inscription

2 .Le fonctionnement d'une auto-école classique : Un Modèle Traditionnel Face aux Enjeux du Numérique :

Malgré l'essor des technologies digitales, les auto-écoles traditionnelles continuent de fonctionner selon un modèle fortement ancré dans des pratiques manuelles. La gestion repose principalement sur des supports papier, des outils analogiques ou des logiciels non connectés entre eux. Bien que ce modèle puisse être perçu comme plus « humain », il entraîne une gestion fragmentée et peu efficace.

La centralisation des données est quasi inexistante : les informations sont dispersées entre différents supports et départements, sans circulation fluide. Cette organisation engendre de nombreux dysfonctionnements en termes de précision, de réactivité et d'efficacité opérationnelle.

Tâches Administratives Manuelles : Un Frein à l'Efficacité

2.1 Inscription des candidats : Une Procédure Lourde et à Risque

L'inscription implique le remplissage de formulaires papier et la remise de nombreux justificatifs (CNI, photos, attestations médicales…). Ces données sont ensuite vérifiées et saisies manuellement, augmentant les risques d'erreurs. L'archivage physique complique l'accès rapide aux dossiers et pose des problèmes de confidentialité.

2.2 Planification des cours : Une Coordination Délicate

Les plannings sont souvent réalisés sur des tableaux blancs ou des agendas papier. L'organisation des leçons doit concilier les disponibilités des candidats, des moniteurs et des véhicules, rendant les ajustements complexes. Les imprévus sont fréquents, et les modifications manuelles peuvent engendrer erreurs et confusions.

2.3 Suivi pédagogique : Une Traçabilité Limitée et Subjective

Chaque moniteur note à la main la progression des candidats dans un carnet ou une fiche. Cette méthode, sujette à l'oubli et au jugement personnel, complique l'évaluation globale et le partage des informations entre formateurs.

2.4 Gestion des moniteurs et véhicules : Une Orchestration Complexe

Le suivi des emplois du temps, qualifications, congés, et de l'affectation des véhicules est réalisé manuellement. L'absence de centralisation rend difficile l'optimisation des ressources et le suivi de la maintenance des véhicules.

2.5 Suivi des paiements : Une Comptabilité Fragile

Les paiements sont enregistrés sur papier ou dans des fichiers Excel, souvent gérés par une seule personne. Cette méthode empêche une vision claire de la situation financière en temps réel, génère des erreurs et ralentit la prise de décision.

2.6 Organisation des examens : Un Processus Peu Flexible

Les inscriptions, convocations et résultats d'examen sont gérés sur papier. La communication est souvent informelle et non traçable, rendant les ajustements de dernière minute complexes et sujets aux erreurs.

2.7 Communication avec les candidats : Informelle et Peu Traçable

Les échanges s'effectuent par appels ou SMS, sans système centralisé. Cela rend difficile la conservation des échanges, le suivi personnalisé ou la transmission fluide d'informations à tous les acteurs concernés.

3. Limites et Contraintes des Systèmes Actuels : Analyse SWOT

Pour mieux cerner les enjeux liés au fonctionnement traditionnel des auto-écoles, une analyse SWOT (forces, faiblesses, opportunités et menaces) permet d'en dégager une vision synthétique et stratégique :

Forces	Faiblesses
Relation client potentiellement perçue comme plus humaine grâce au contact direct.	Manque de centralisation des informations : données dispersées, gestion et consultation difficiles.
Connaissance empirique des processus.	Risques d'erreurs humaines : saisie de données, planification, suivi des paiements.
Fidélisation des clients par le bouche-à-oreille local.	Difficulté de synchronisation et de mise à jour : plannings et informations non actualisés en temps réel, entraînant des incohérences.
Procédures familières pour le personnel, réduisant les besoins de formation.	Perte d'informations et archivage complexe : documents papier perdus, endommagés ou mal classés, suivi à long terme difficile.
Expérience de certains membres du personnel, apportant un savoir-faire non formalisé.	Communication inefficace : systèmes non centralisés et non automatisés, gestion informelle et peu fiable des échanges.
	Temps administratif important : traitement manuel des tâches chronophage.
	Manque de visibilité financière en temps réel.
	Difficulté d'optimisation des ressources (moniteurs, véhicules).

Opportunité	Menaces
Digitalisation des processus pour une efficacité accrue.	Concurrence accrue des auto-écoles ayant déjà opéré leur transformation numérique.
Amélioration de la satisfaction des candidats et des moniteurs.	Inefficacité et erreurs coûteuses dues au maintien de systèmes obsolètes.
Nouveaux services possibles grâce à la digitalisation (e-learning, suivi en ligne).	Perte d'attractivité auprès d'une clientèle de plus en plus connectée.
Optimisation de la gestion des ressources et réduction des coûts.	Risques de non-conformité réglementaire liés à une gestion des données non sécurisée ou non centralisée
Communication plus fluide et personnalisée avec les candidats.	Difficulté à s'adapter aux évolutions technologiques et aux attentes du marché.
Meilleure analyse des données pour la prise de décision.	Démotivation du personnel face à des processus

4. Conclusion : Une Transformation Numérique Indispensable

L'analyse du fonctionnement traditionnel met en évidence les nombreuses limites liées à l'utilisation d'outils manuels et non intégrés : tâches chronophages, erreurs fréquentes, difficultés de communication et absence de visibilité en temps réel.

Face à ces constats, la digitalisation s'impose comme une réponse incontournable. Une solution numérique centralisée permettrait non seulement d'automatiser les tâches et d'améliorer la coordination, mais aussi de répondre aux attentes croissantes des candidats et aux exigences d'une gestion moderne. Il est donc temps pour les auto-écoles de franchir le cap de la transformation digitale pour gagner en performance, en réactivité et en qualité de service.

CHAPITRE 2 :

L'ANALYSE DES BESOINS

La transition d'un système de gestion artisanal vers une solution numérique performante ne s'improvise pas. Tout comme un architecte conçoit les plans d'un immeuble avant sa construction, la modélisation logicielle exige une réflexion structurée pour éviter l'improvisation et les erreurs coûteuses. Ce chapitre incarne cette démarche méthodique : il ne s'agit pas seulement de lister des besoins, mais de construire un model cohérent qui servira de socle au développement.

L'analyse des besoins, couplée à la programmation orienté objet (POO), révèle l'ampleur du travail conceptuel nécessaire. Définir des classes, leurs relations, leurs attributs et leurs méthodes n'est pas une simple formalité technique : c'est une étape cruciale pour traduire les réalités métiers (candidats, moniteurs, examens…) en entités logicielles interactives. UML (Unified Modeling Language), norme incontournable du génie logiciel, offre justement le langage visuel et standardisé pour formaliser cette complexité.

Ce chapitre répond à un double impératif :

Identifier les attentes fonctionnelles et techniques des utilisateurs : en s'appuyant sur les dysfonctionnements exposés au chapitre 1.

Poser les bases d'une modélisation rigoureuse grâce à UML : qui permettra de conceptualiser l'application avant toute ligne de code.

À travers des diagrammes, nous passerons progressivement du **pourquoi** (les besoins) au **comment** (l'architecture logicielle). Cette approche prévient les écueils classiques du développement et garantit une solution alignée sur les enjeux opérationnels, réglementaires et humains des auto-écoles.

1. Recueil des besoins fonctionnels et non fonctionnels :

Le recueil des besoins constitue une étape cruciale dans tout projet informatique. Il permet de définir ce que le système devra faire (besoins fonctionnels) et les exigences qu'il devra respecter pour bien fonctionner (besoins non fonctionnels). Cette phase s'appuie sur les résultats de l'étude de l'existant, les observations sur le terrain, les entretiens avec les utilisateurs finaux (gestionnaires, moniteurs, candidats) ainsi que les objectifs stratégiques du projet.

1.1. Besoins fonctionnels :

Les besoins fonctionnels correspondent aux fonctionnalités que l'application doit proposer. Ils traduisent les services attendus par les différents types d'utilisateurs.

a. Gestionnaires (administration de l'auto-école) :

- Créer, modifier, supprimer les dossiers des candidats.
- Gérer les inscriptions et les pièces justificatives.
- Planifier les séances de conduite et de code.
- Attribuer les moniteurs et les véhicules.
- Gérer les examens (prévision, résultats).

- Suivre les paiements et l'historique financier des candidats.
- Gérer les comptes utilisateurs (création, activation, désactivation).
- Consulter des statistiques globales sur les performances des élèves et moniteurs.

b. Moniteurs :

- Consulter leur emploi du temps.
- Accéder à la fiche pédagogique de chaque candidat.
- Consulter les résultats des candidats aux examens.
- Consulter leur planning personnel
- Accéder à sa fiche pédagogique personnelle.

c. Candidats :

- Consulter leur planning personnel (séances, examens).
- Accéder à des ressources pédagogiques (vidéos, documents PDF[code]).
- Consulter leur situation financière (paiements effectués/restants).
- Accéder à sa fiche pédagogique personnelle.
- Consulter le résultat de ses examens.

d. Fonctions transversales :

- Authentification sécurisée des utilisateurs selon leur rôle (candidat, moniteur, gestionnaire).
- Interface de recherche et de filtrage des données (élèves, séances, moniteurs).
- Export de données (PDF) pour archivage ou impression.
- Sauvegarde régulière de la base de données.

1.2. Besoins non fonctionnels :

Les besoins non fonctionnels définissent les qualités que doit posséder le système. Ils touchent à la performance, la sécurité, la fiabilité, l'ergonomie ou encore la portabilité.

a. Performance :

- Le temps de chargement des pages doit rester inférieur à 2 secondes.
- L'application doit pouvoir gérer simultanément plusieurs utilisateurs sans ralentissement (scalabilité).

b. Sécurité :

- Les données personnelles doivent être protégées par des mécanismes de cryptage (hachage des mots de passe, accès HTTPS) :

    - **Le hachage des mots de passe** : au lieu de stocker les mots de passe en clair, on les transformer en une valeur illisible pour que personne ne puisse les lire, même en cas de piratage.
    - **L'accès HTTPS** : si l'application communique via Internet, HTTPS permet de chiffrer les échanges entre l'utilisateur et le serveur, empêchant les interceptions.

- Les rôles et permissions doivent être strictement définis et respectés.

c. Fiabilité :

- Le système doit garantir une disponibilité élevée
- Les données doivent être sauvegardées automatiquement pour éviter toute perte.

d. Ergonomie :

Les 10 heuristiques de Nielsen :

- **Visibilité du statut du système** : Le système doit toujours informer l'utilisateur de ce qui se passe.
Ex : Lorsqu'on télécharge un fichier, une barre de progression indique l'avancement.

- **Correspondance entre le système et le monde réel** : Utiliser un langage et des concepts familiers à l'utilisateur, proches de la réalité.
Ex: Afficher <Retour> pour le retour, plutôt que des termes techniques.

- **Contrôle et liberté de l'utilisateur** : Permettre à l'utilisateur d'annuler ou de revenir en arrière facilement.
Ex : Un bouton « Annuler » dans un formulaire pour revenir en arrière.

- **Cohérence et standards**
Respecter les conventions courantes pour que l'utilisateur comprenne rapidement l'interface.
Ex : La loupe symbolise toujours la fonction recherche.

- **Prévention des erreurs**
Éviter que l'utilisateur puisse faire des erreurs en concevant bien l'interface.
Ex : Demander une confirmation avant de supprimer un fichier.

- **Reconnaissance plutôt que souvenir :**
L'utilisateur ne doit pas avoir à mémoriser des informations, tout doit être visible ou facilement accessible.
Ex: Proposer un menu déroulant plutôt qu'un champ à remplir à la main.

- **Flexibilité et efficacité d'utilisation**
Offrir des raccourcis pour les utilisateurs expérimentés sans gêner les débutants.
Exemple : Raccourcis clavier comme Ctrl+C pour copier

- **Esthétique et design minimaliste**
L'interface doit être claire, sans éléments inutiles qui gênent la compréhension.
Ex : Une page avec peu de boutons et un design épuré.

- **Aide à la reconnaissance, diagnostic et récupération d'erreurs**
Les messages d'erreur doivent être clairs et guider l'utilisateur pour corriger le problème.
Ex: « Mot de passe incorrect, veuillez réessayer » au lieu d'un message vague.

- **Aide et documentation**

Proposer une aide facile à trouver si l'utilisateur rencontre des difficultés.
Ex : Un bouton « Aide »

Les règles d'or de Coutaz :

- **Lutter pour la cohérence**

Le système doit fonctionner de façon homogène dans toutes ses parties.
Ex : Les mêmes boutons doivent avoir les mêmes fonctions et la même apparence partout dans l'application.

- **Lutter pour la concision**

L'information doit être claire, utile et réduite à l'essentiel.
Ex **:** Éviter les phrases longues ou inutiles dans les menus et messages.

- **Réduire la charge cognitive**

L'utilisateur ne doit pas avoir à mémoriser trop d'informations à la fois.
Ex **:** Afficher les instructions ou options visibles plutôt que demander à l'utilisateur de s'en souvenir.

- **Mettre le contrôle entre les mains de l'utilisateur**

L'utilisateur doit pouvoir agir librement, annuler une action, revenir en arrière.
Ex **:** Bouton "Annuler" sur un formulaire, ou possibilité de modifier une réservation.

- **Souplesse d'utilisation**

L'interface doit convenir à différents types d'utilisateurs, novices ou experts.
Ex **:** Proposer des raccourcis clavier pour les utilisateurs avancés tout en gardant les menus clairs pour les débutants.

- **Structurer le dialogue**

Organiser les interactions de façon logique et progressive.
Ex **:** Suivre des étapes claires dans un processus d'inscription (étape 1, 2, 3…).

- **Prévoir et prévenir les erreurs**

Le système doit éviter les erreurs et aider l'utilisateur à les corriger.
Ex **:** Afficher un message si un champ obligatoire est vide avant de valider un formulaire.

e. Portabilité et accessibilité :

- L'application doit être accessible via un navigateur web, sans installation nécessaire.

f. Maintenabilité et évolutivité :

- Le code doit être documenté et structuré pour permettre des mises à jour futures (ajout de fonctionnalités, corrections).
- L'architecture choisie doit permettre l'ajout ultérieur de modules .

2. Acteurs du système

Dans le cadre de l'application de gestion d'une auto-école, plusieurs types d'utilisateurs interagissent avec le système selon leurs rôles et responsabilités. Chaque acteur dispose d'un accès personnalisé à l'application, avec des fonctionnalités spécifiques adaptées à ses besoins. Les principaux acteurs sont : l'administrateur, le moniteur et le candidat.

2.1. **Administrateur**

Responsable de la gestion et du contrôle complet du système.

2.2. **Moniteur**

Chargé de l'encadrement et du suivi des candidats dans leur parcours.

2. 3. **Candidat**

Utilisateur final qui accède aux services liés à sa formation et son suivi.

3. Les cas d'utilisations principaux :

1. **Acteur principal** : Administrateur

Objectif : L'administrateur se connecte à l'application via un identifiant et mot de passe, puis accède à une interface centralisée lui permettant de gérer les comptes utilisateurs, candidats, moniteurs, véhicules, ainsi que de planifier les séances et examens et de gérer et consulter les fiches et résultats.

Pré conditions :
L'administrateur doit être connecté au système avec des identifiants valides.

Post conditions :
Les informations relatives aux utilisateurs, candidats, moniteurs, véhicules, plannings ou résultats sont mises à jour, enregistrées ou consultées avec succès dans le système.

Ses cas d'utilisation principaux :

- Authentification : - Saisir le login
  - Saisir le mot de passe
- Gestion des comptes Utilisateur

- Gestion des candidats : - Ajouter candidat.
  -Modifier candidat.
  - Supprimer candidat.
  - Rechercher candidat.
- Gestion des moniteurs : -Ajouter moniteur.
  -Modifier moniteur.
  - Supprimer moniteur.
  - Rechercher moniteur.
- Gestion des véhicules : -Ajouter véhicule.
  -Modifier véhicule.
  - Supprimer véhicule.
  - Rechercher véhicule.
- Planifier séance
- Planifier examen
- Consulter fiche candidat
- Consulter fiche moniteur
- Consulter planning examen /séance
- Consulter résultat examen

2 .Acteur principal : Moniteur

Objectif : Permettre au moniteur de s'authentifier, consulter ses informations personnelles, accéder aux plannings des séances, rechercher un candidat, consulter sa fiche et ses résultats d'examen pour mieux suivre sa progression.

Pré conditions :

- L'administrateur dispose d'un compte valide (identifiant et mot de passe).
- La base de données contient des informations sur les candidats, leurs séances et leurs examens.

Post conditions :

- Le système affiche les données demandées (fiche, planning, résultats).
- Aucune modification n'est apportée aux données (consultation uniquement).

Ses cas d'utilisation principaux :

- S'authentifier en saisissant son identifiant et mot de passe.
- Consulter sa fiche personnelle
- Accéder au planning des séances (heures, dates, candidats).
- Rechercher un candidat par nom et prénom
- Consulter la fiche du candidat
- Consulter les résultats d'examen du candidat.

3. Acteur principal: Candidat

Objectif :

Permettre au candidat de s'authentifier, consulter ses informations personnelles, accéder à son planning d'examen et de séances, ainsi que consulter ses résultats d'examen.

Pré conditions :

Le candidat doit être inscrit dans le système et posséder des identifiants valides pour accéder à l'application.

Post conditions :

Le candidat a accès à ses informations personnelles, à ses plannings et à ses résultats, sans pouvoir les modifier.

Ses cas d'utilisation principaux :

- S'authentifier en saisissant son identifiant et mot de passe.
- Consulter sa fiche personnelle
- Consulter le planning des séances
- Consulter le planning des examens
- Consulter les résultats d'examen

✓ ***Interaction avec la base de données*** *: Chaque action déclenche une opération (lecture, écriture, modification ou suppression) sur les données stockées dans le système.*

4. Les diagrammes UML :

Pour modéliser le système et illustrer son fonctionnement, plusieurs diagrammes UML ont été réalisés. Ces diagrammes permettent de visualiser les interactions entre les acteurs et le système, ainsi que la structure et le comportement de l'application.

4.1 Diagramme de contexte :

Le diagramme de contexte représente une vue globale du système de gestion de l'auto-école et des interactions qu'il entretient avec les différents acteurs externes. Il permet de visualiser les échanges d'informations entre le système et ses utilisateurs, en mettant en évidence les flux entrants et sortants.

Le système centralisé est entouré de trois acteurs principaux : l'administrateur, le moniteur et le candidat. Chacun interagit avec le système selon ses besoins et ses droits d'accès. Le diagramme ne montre pas le détail des traitements internes, mais uniquement les flux d'informations échangés entre les utilisateurs et le système**.**

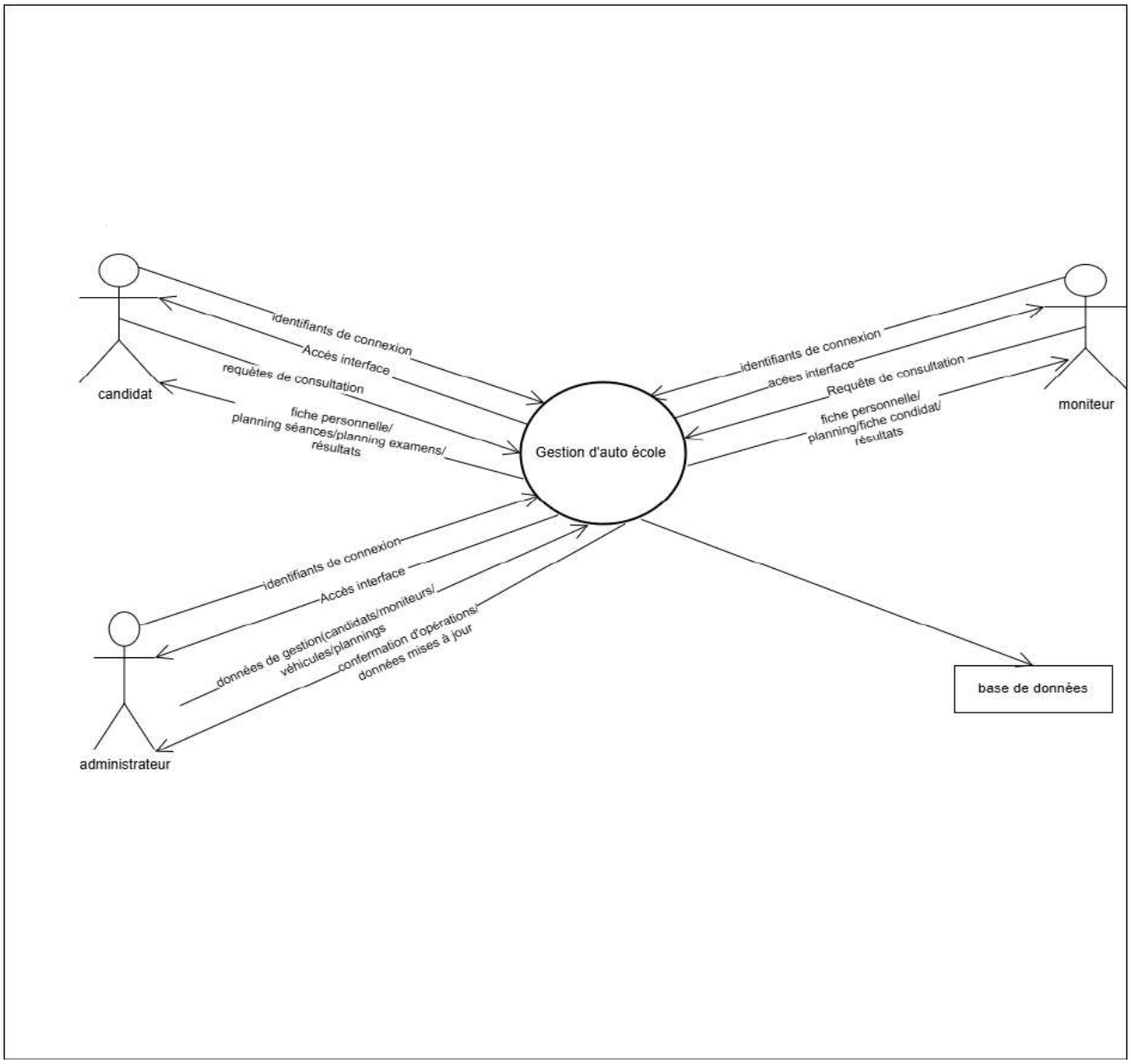


Figure 01 : « Diagramme de Contexte»

4.2 Diagrammes des cas d'utilisations :

Le diagramme de cas d'utilisation est un outil visuel qui sert à représenter les interactions entre les utilisateurs (appelés acteurs) et le système. Il montre de manière claire et synthétique les différentes fonctionnalités offertes par le système et comment chaque acteur peut les utiliser.

Ce diagramme permet de comprendre rapidement les besoins du système en se concentrant sur ce que les utilisateurs veulent faire, sans entrer dans les détails techniques.

Il est très utile pour organiser le travail de développement et garantir que le système répond bien aux attentes des utilisateurs.

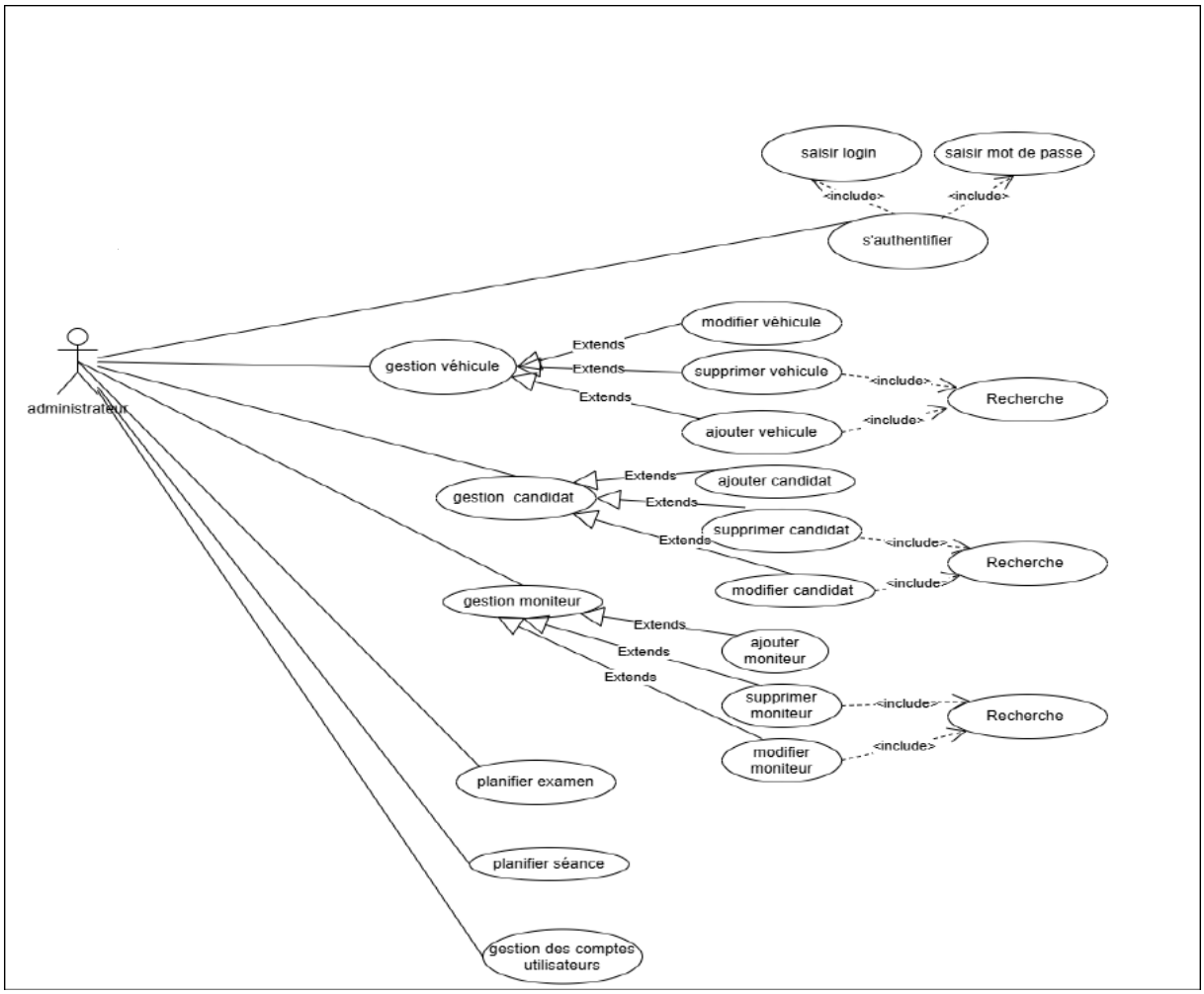


Figure 02 : «Diagramme de Cas d'utilisation : Acteur Administrateur »

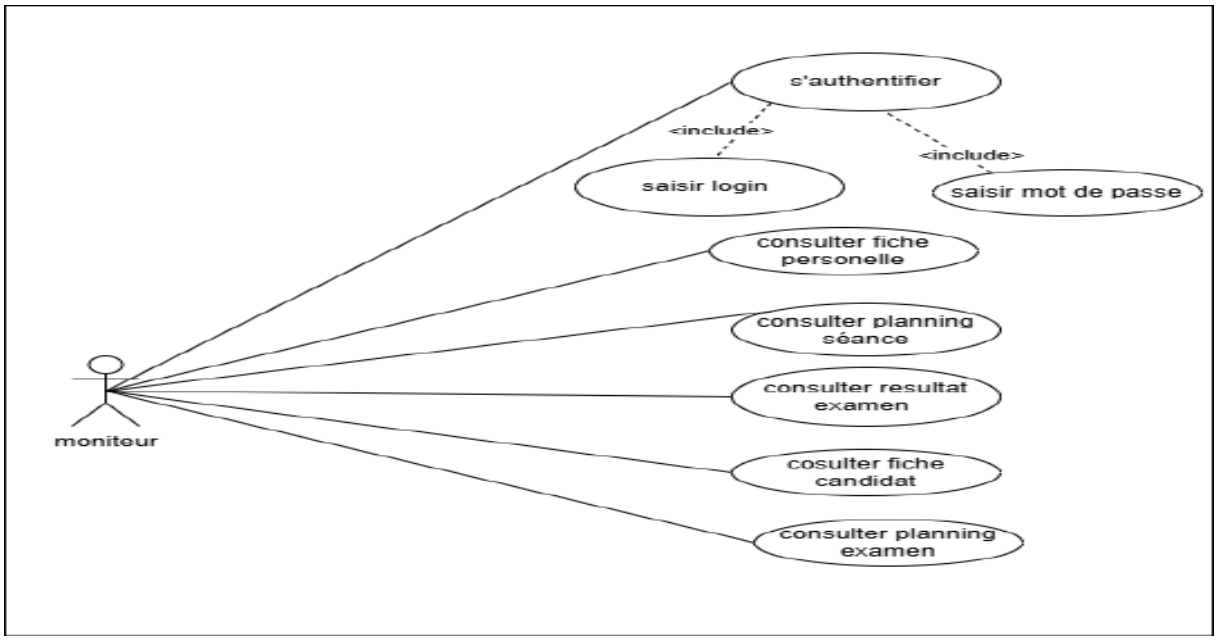


Figure 03 : «Diagramme de Cas d'utilisation : Acteur Moniteur »

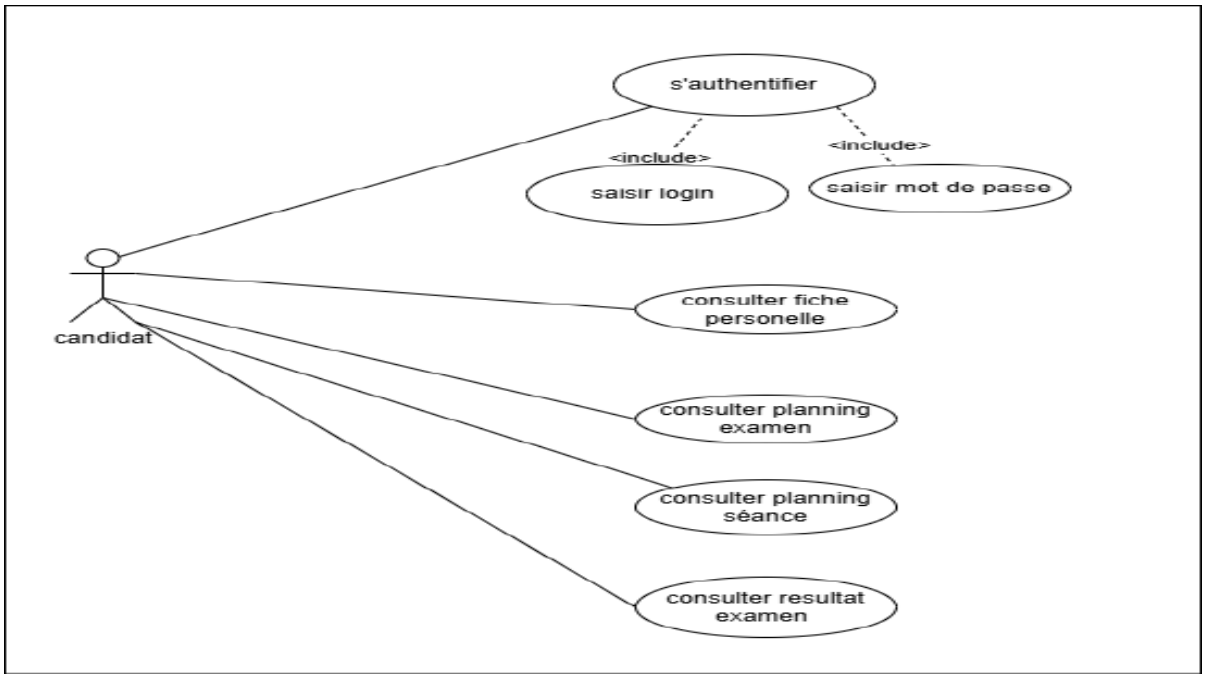


Figure 04 : «Diagramme de Cas d'utilisation : Acteur Candidat »

4.3 Diagrammes de séquences :

Le diagramme de séquence illustre la manière dont les objets du système interagissent dans le temps pour réaliser une fonctionnalité spécifique. Il permet de visualiser l'enchaînement chronologique des messages échangés entre les acteurs et le système.

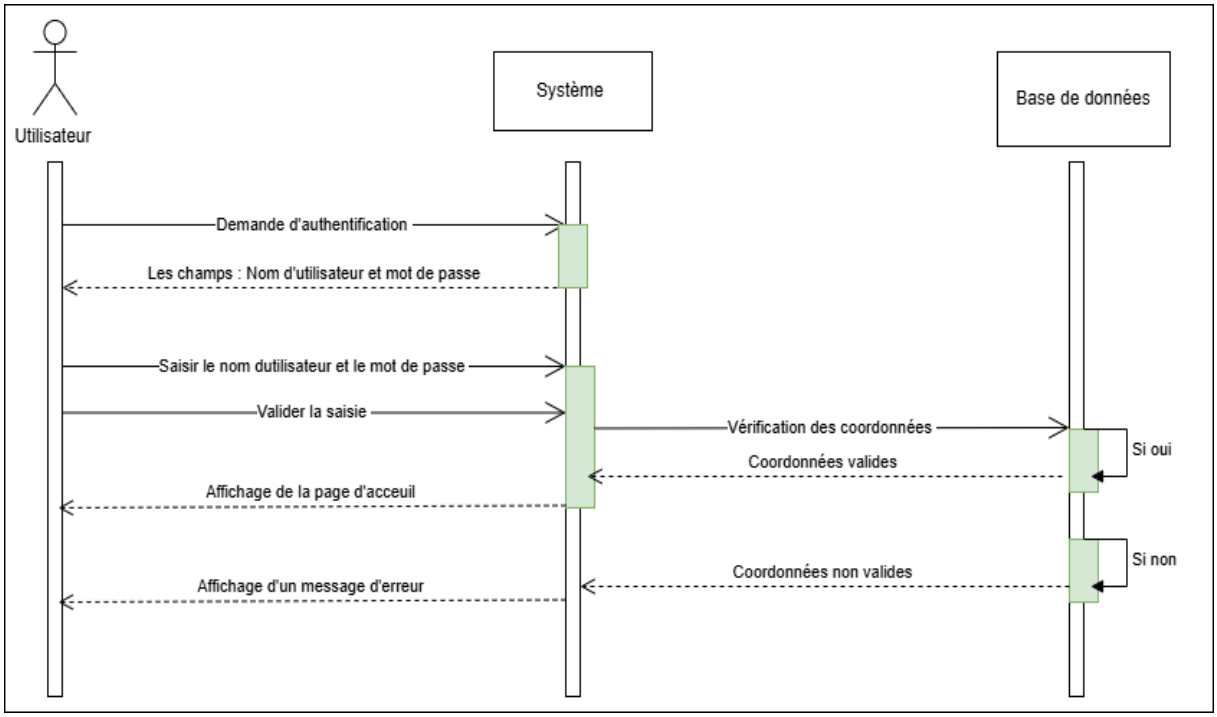


Figure 05 : «Diagramme de séquence-Authentification- »

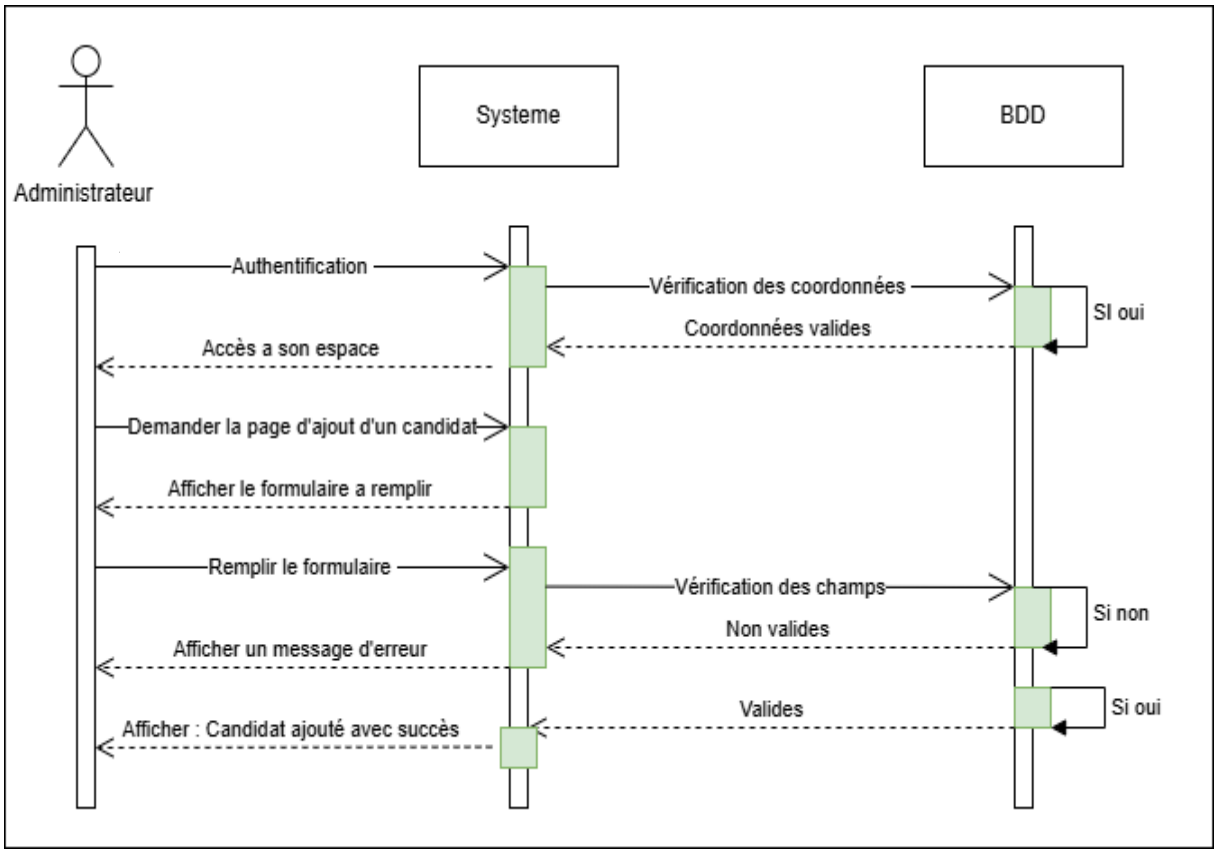


Figure 06 : «Diagramme de séquence-Ajout d'un candidat- »

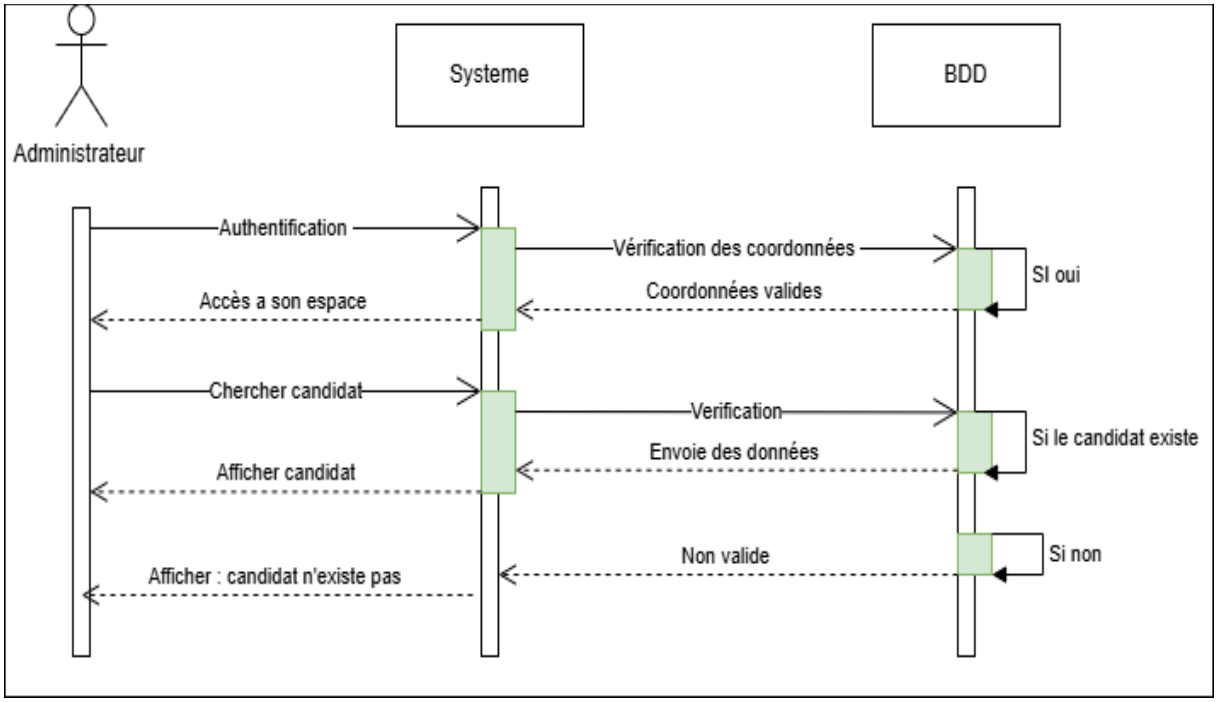


Figure 07 : «Diagramme de séquence-Recherche- »

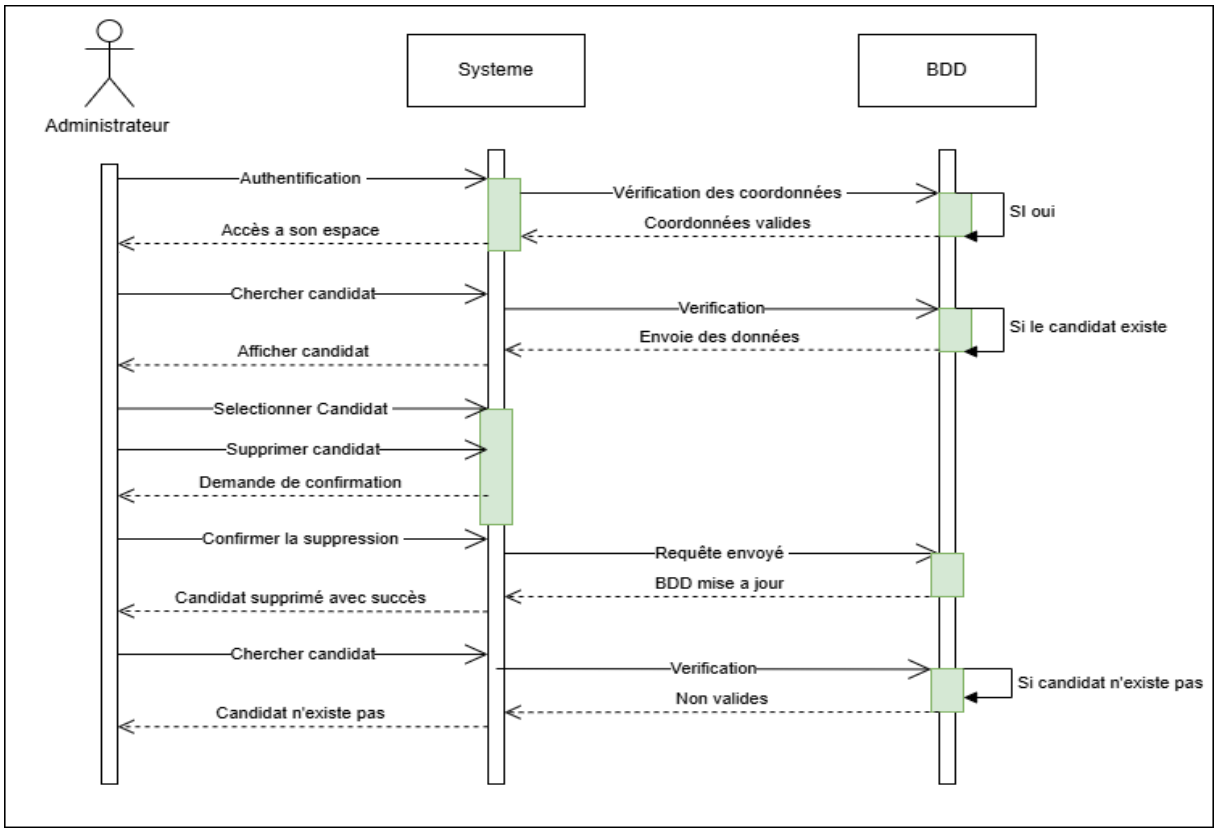


Figure 08 : «Diagramme de séquence-Suppression d'un candidat- »

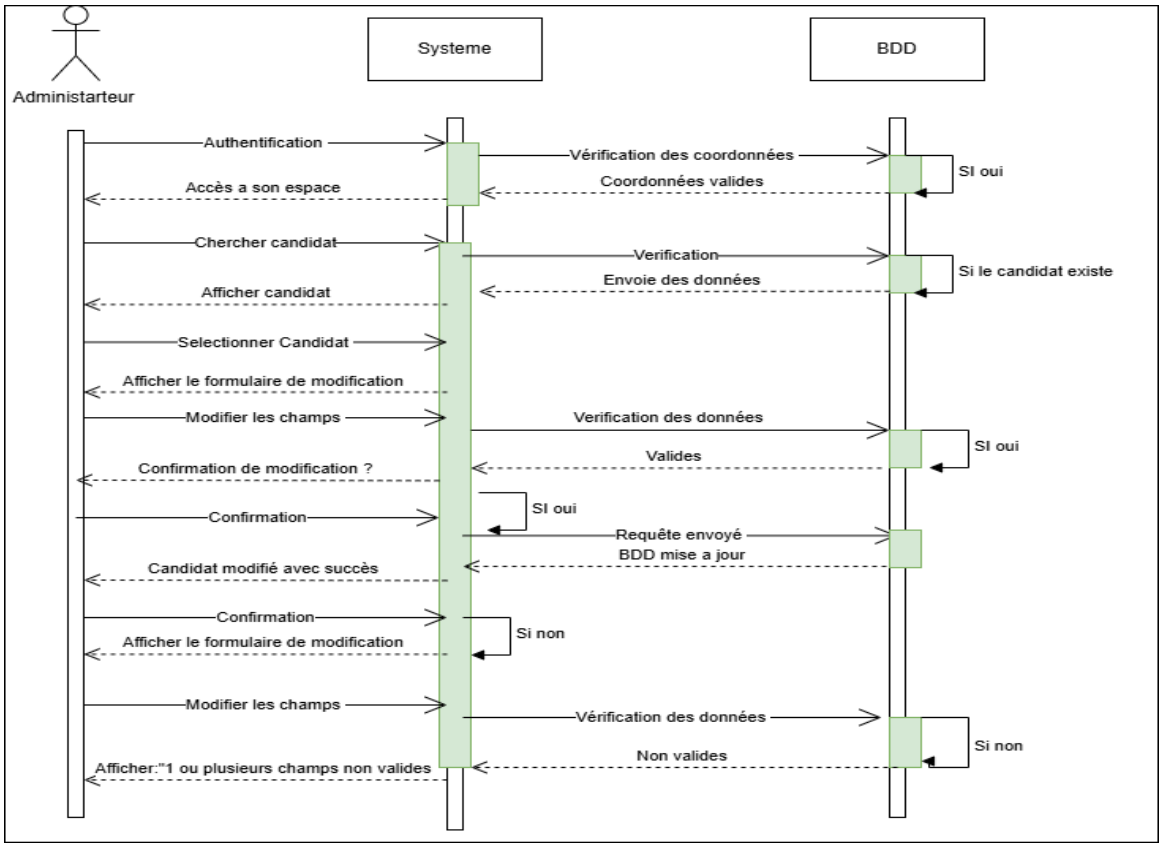


Figure 09: «Diagramme de séquence-Modification d'un candidat- »

- ✓ **Les opérations d'ajout, de recherche, de modification et de suppression d'un moniteur ou d'un véhicule suivent la même logique fonctionnelle que celles illustrées dans les figures 6, 7, 8 et 9.**

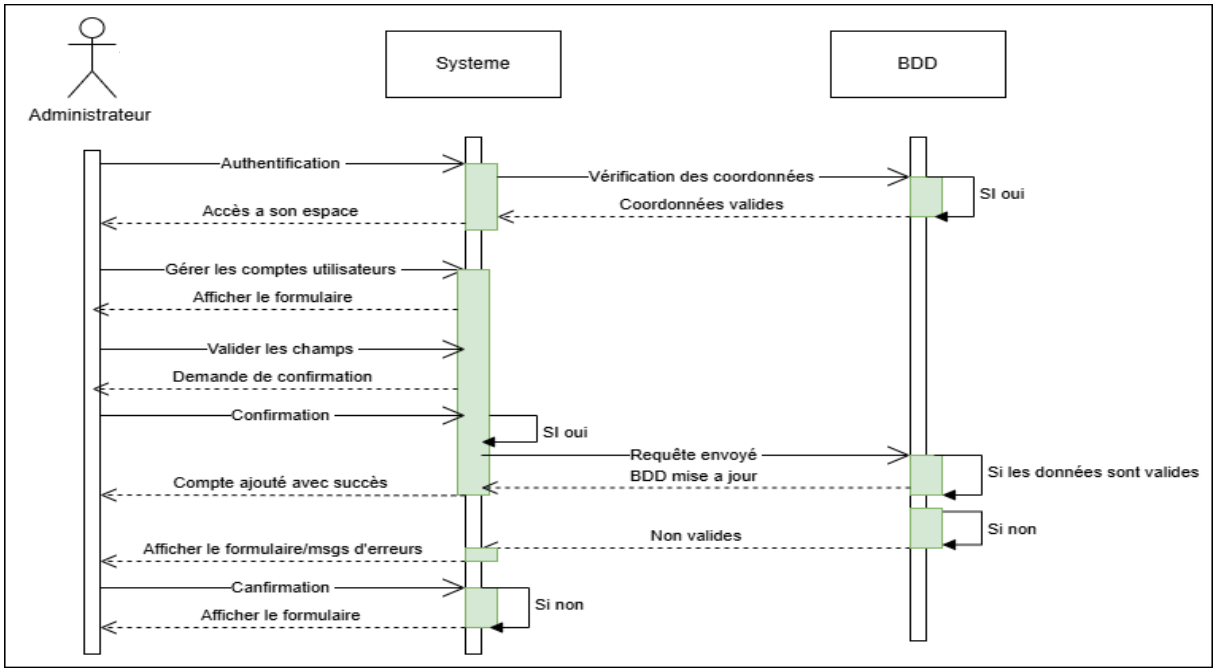


Figure 10: «Diagramme de séquence-Gérer les comptes utilisateurs »

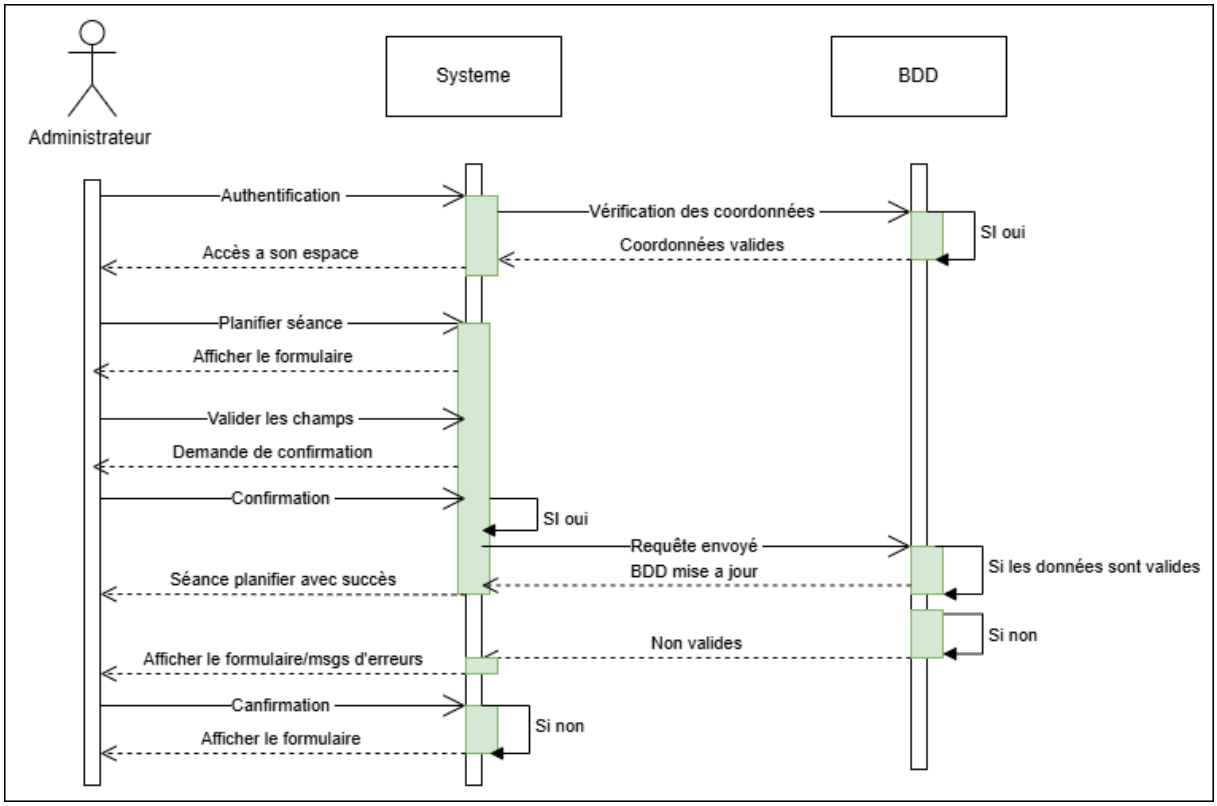


Figure 11: «Diagramme de séquence-Planifier séances »

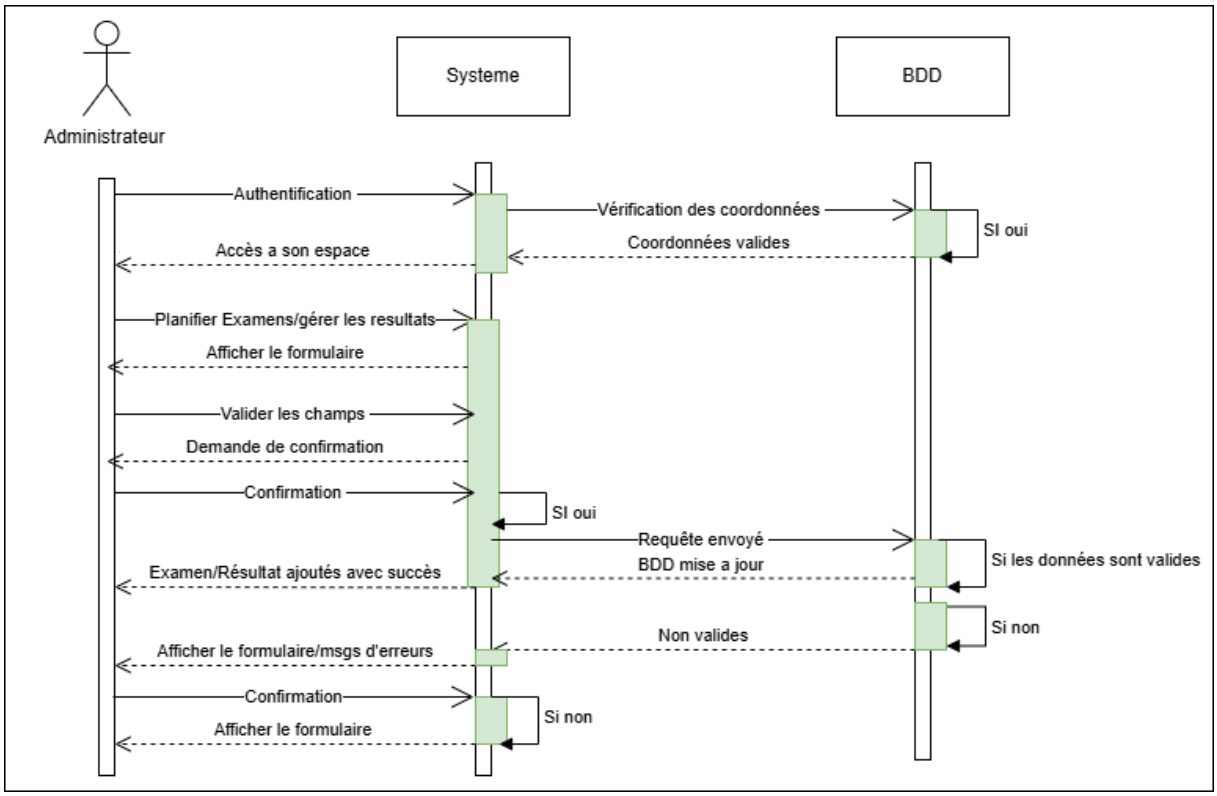


Figure 12: «Diagramme de séquence-Planifier examen/Résultat examens »

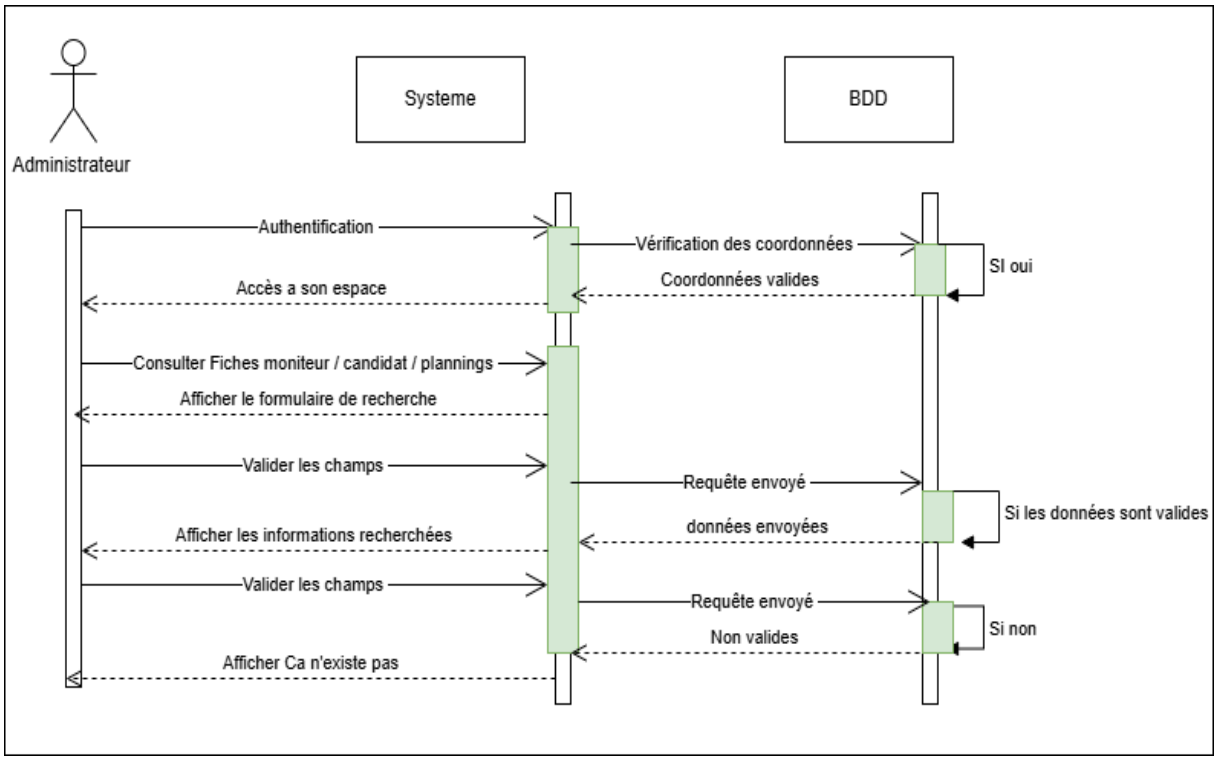


Figure 13: «Diagramme de séquence-Consulter fiches personnelles moniteurs/ candidat / plannings examens / séances »-admin-

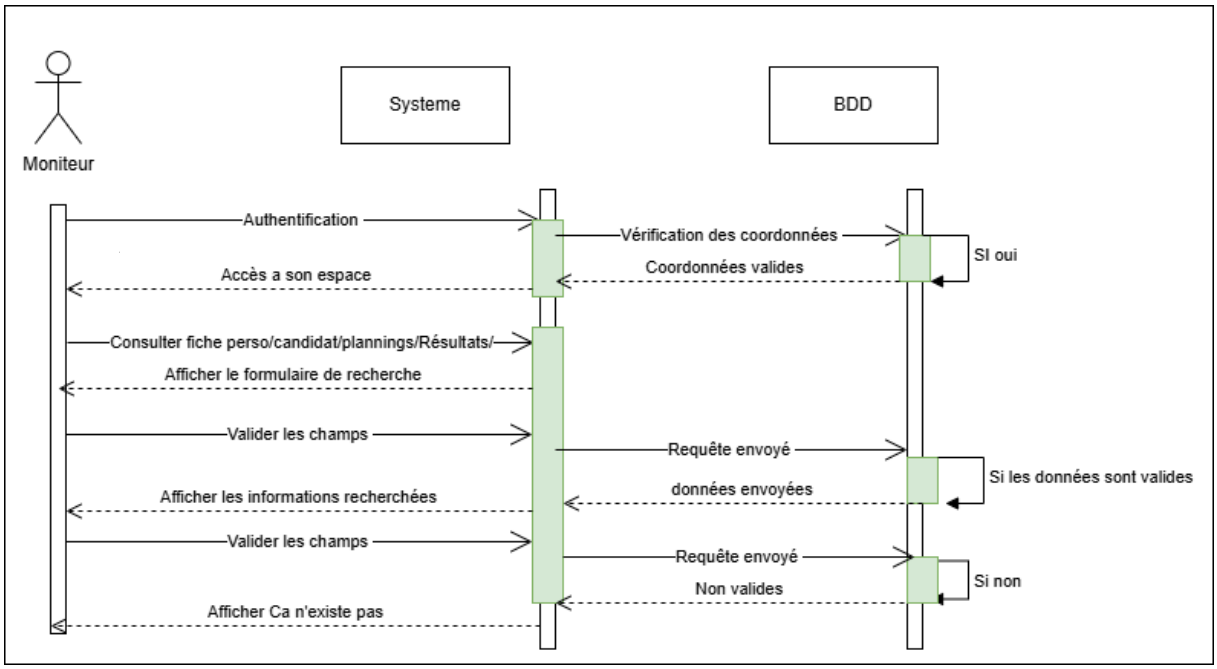


Figure 14: «Diagramme de séquence-Consulter fiches personnelles candidat / plannings examens / séances /Résultats »-moniteurs-

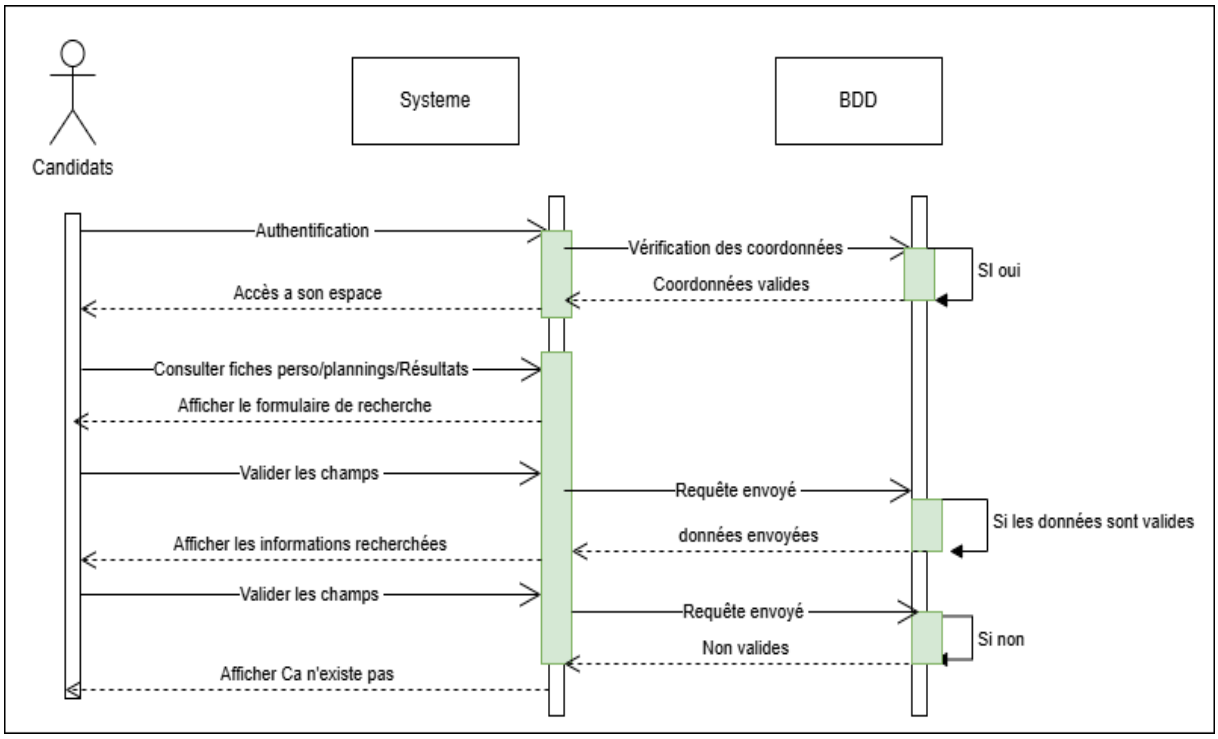


Figure 15: «Diagramme de séquence-Consulter fiches personnelles / plannings examens / séances /Résultats »-candidats-

4.4 Diagramme de classes :

Le diagramme de classes est un élément central de la modélisation orientée objet. Il permet de représenter la structure statique du système en définissant les classes, leurs attributs, leurs méthodes, ainsi que les relations entre elles (association, héritage, agrégation…).

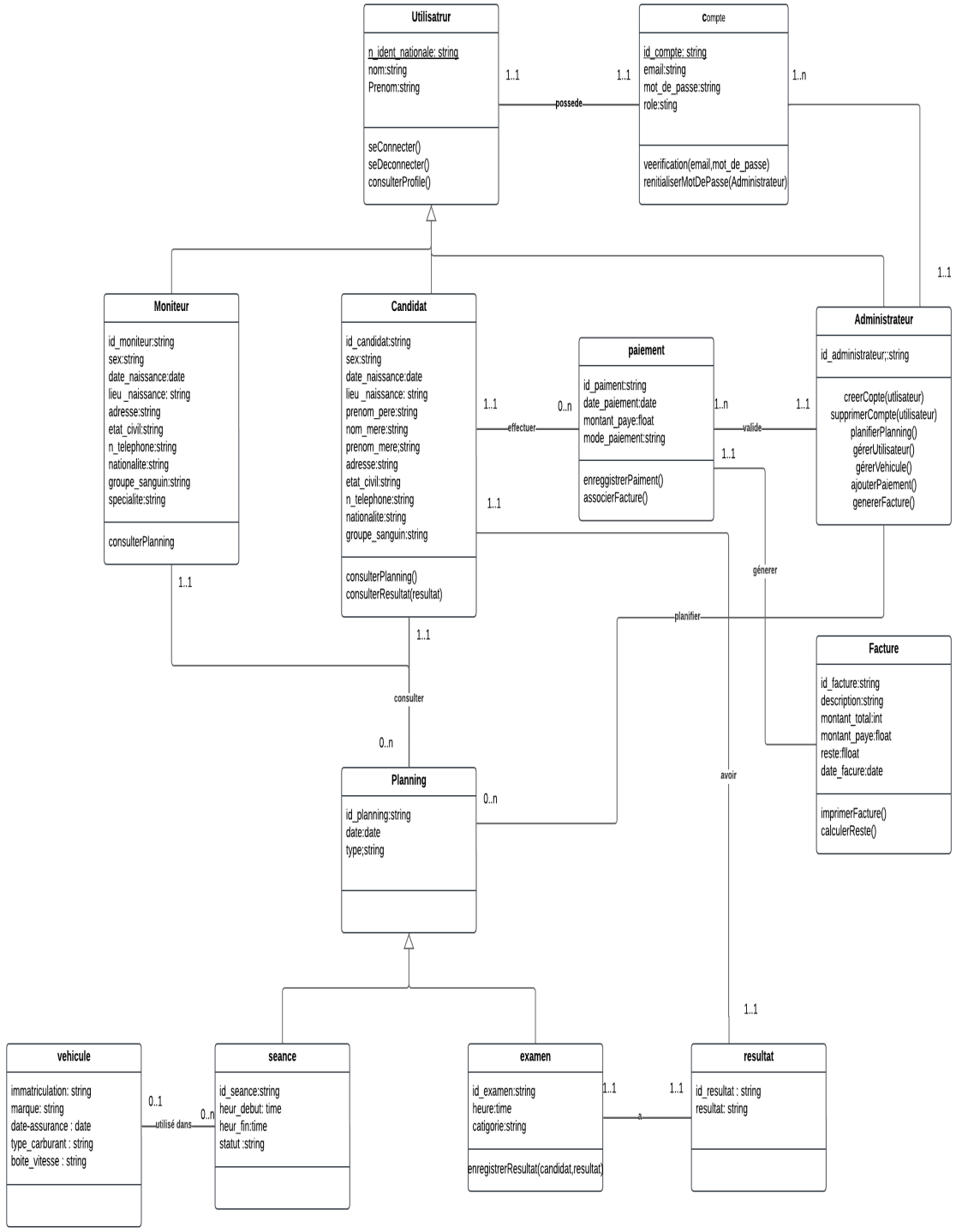


Figure 16 : «Diagramme de classes »

4.5 Diagramme de déploiement :

Le diagramme de déploiement ci-dessus représente l'architecture physique de l'application web de gestion d'auto-école. Il repose sur une architecture client-serveur. Les utilisateurs accèdent à l'application via un navigateur web (Client Web), qui communique avec un serveur web Python (Django). Ce dernier gère la logique métier et interagit avec une base de données SQL distante, assurant ainsi la fiabilité du système.

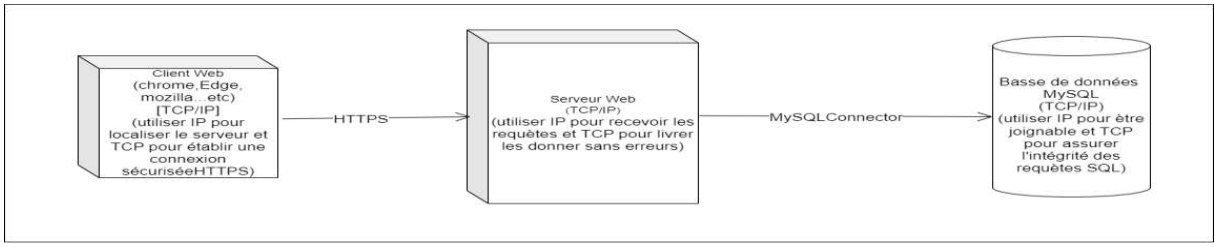


Figure 17 : «Diagramme de déploiement »

Conclusion : Vers une Conception Structurée et Efficace

Ce chapitre a posé les fondations essentielles pour la réussite du projet en insistant sur l'importance d'une modélisation rigoureuse avant le développement. Le recueil précis des besoins fonctionnels et non fonctionnels a permis d'identifier clairement les attentes des différents utilisateurs, garantissant ainsi une solution adaptée aux réalités métier des auto-écoles. La définition des acteurs et de leurs cas d'utilisation a structuré les interactions avec le système, tandis que l'utilisation d'UML offre un langage commun pour visualiser et valider la conception. En combinant une analyse méthodique avec une modélisation orientée objet, ce travail prépare efficacement le passage à la phase de développement, minimisant les risques d'erreurs et maximisant la cohérence et la maintenabilité de l'application.

CHAPITRE 3 :

CONCEPTION DE LA SOLUTION

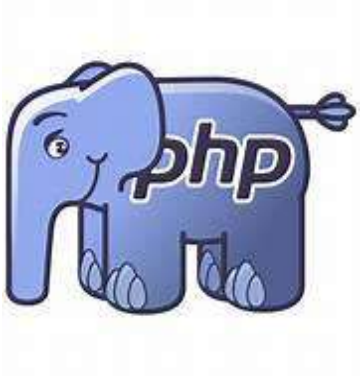
À l'image d'un ingénieur qui sélectionne avec rigueur ses matériaux et ses outils avant de construire un ouvrage, le développeur pose les bases techniques d'une application en choisissant les langages, les Framework, les bases de données et les environnements adaptés. Cette phase de conception est essentielle pour garantir la solidité, la cohérence et l'efficacité de la solution à développer.

Ce chapitre présente les fondations de l'application : les choix technologiques adoptés, l'architecture logicielle retenue (modèle MVC et approche client-serveur), la conception de la base de données, ainsi que les premières maquettes des interfaces utilisateur. Ces éléments constituent le socle technique sur lequel reposera l'ensemble du développement.

1. Les choix technologiques :

Dans ce projet, plusieurs technologies ont été utilisées pour développer une application web complète. Le langage PHP a été utilisé côté serveur pour gérer la logique métier . Pour l'interface utilisateur, HTML, CSS et JavaScript ont permis de créer des pages dynamiques et interactives. La base de données MySQL, accessible via phpMyAdmin sous XAMPP, a été choisie pour stocker les données. L'ensemble du développement a été réalisé avec Visual Studio Code, un éditeur adapté au développement web.

1.1 Le langage PHP:



Le langage PHP (PHP: Hypertext Preprocessor) a été créé en 1994 par Rasmus Lerdorf pour faciliter la création de pages web dynamiques. À l'origine, PHP signifiait Personal Home Page, avant de devenir un acronyme récursif reflétant son rôle de préprocesseur d'hypertexte.

Simple à utiliser et bien intégré au HTML, il s'est imposé comme un langage essentiel du développement web côté serveur. Les versions récentes, comme PHP 7 et PHP 8, ont renforcé ses performances et sa sécurité. Grâce à sa compatibilité avec MySQL et son écosystème riche, PHP reste un outil privilégié pour créer des applications web efficaces.

1.2 HTML :



Créé en 1991 par Tim Berners-Lee, HTML (HyperText Markup Language) permet de structurer le contenu d'une page web. Dans ce projet, il est utilisé pour définir l'ossature de l'interface utilisateur de l'application web, comme les menus, formulaires ou tableaux.

1.3 CSS :



Introduit en 1996 par le W3C, CSS (Cascading Style Sheets) est utilisé pour styliser les pages HTML. Il permet de personnaliser l'apparence de l'interface (couleurs, polices, dispositions) afin d'offrir une expérience visuelle claire et agréable aux utilisateurs de l'auto-école.

1.4 JavaScript :



Créé en 1995 par Brendan Eich, JavaScript permet de rendre les pages web dynamiques et interactives. Dans cette application, il est utilisé pour gérer les actions de l'utilisateur (clics, validations de formulaires...) et actualiser le contenu sans recharger toute la page.

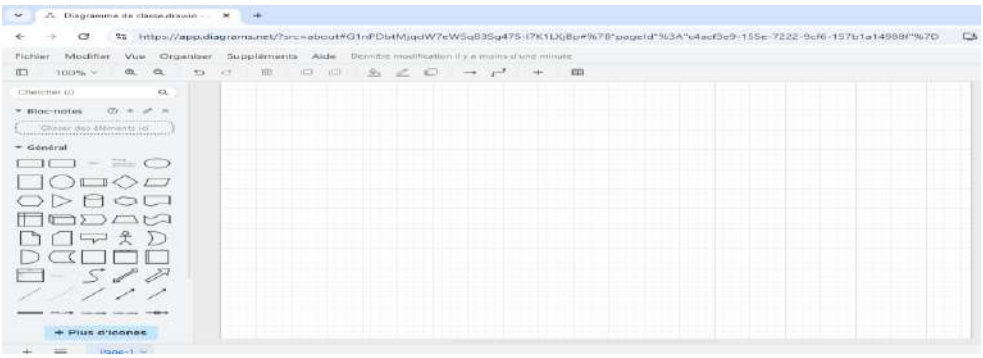
1.5 SQL :



SQL (Structured Query Language) est un langage né dans les années 1970 chez IBM, basé sur le modèle relationnel d'Edgar F. Codd. Il a été conçu pour manipuler et interroger des bases de données relationnelles. D'abord appelé SEQUEL, il devient SQL et est standardisé en 1986. Depuis, il s'est imposé comme le langage de référence pour la gestion des bases de données dans de nombreux systèmes comme MySQL, Oracle ou PostgreSQL.

1.6 Outil de modélisation : Draw.io

Draw.io est un outil en ligne gratuit qui permet de créer des diagrammes et des schémas. Il offre la possibilité de concevoir et de visualiser divers types de diagrammes, tels que des diagrammes de flux, des organigrammes, des diagrammes UML, ainsi que bien d'autres formats de représentation graphique.



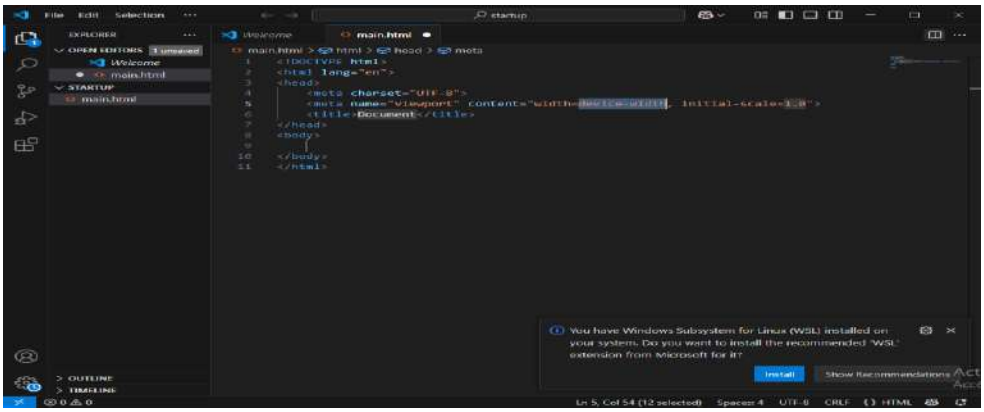
1.7 Serveur local : XAMPP

XAMPP est un ensemble de logiciels open-source qui permet de créer un serveur web local sur un ordinateur, facilitant ainsi le développement de sites web. Il regroupe des programmes essentiels tels qu'Apache, MySQL, PHP et Perl. Dans notre cas, XAMPP a été utilisé pour exécuter le serveur Apache localement et gérer la base de données MySQL via phpMyAdmin, une interface web conviviale.



1.8 Environnement de développement : Visual Studio Code (VS Code)

Visual Studio Code est un éditeur de code open-source développé par Microsoft, conçu pour le développement d'applications web et mobiles. Il est personnalisable, offre des outils avancés de traitement de texte et de nombreuses extensions pour améliorer la productivité des développeurs, tout en prenant en charge plusieurs langages de programmation.



2 Architecture Client-serveur :

L'application repose sur une architecture client-serveur classique :

Le Client : Il s'agit du navigateur web de l'utilisateur (élève, moniteur ou administrateur). Il envoie des requêtes HTTP/HTTPS via une interface web construite en HTML, CSS et JavaScript, afin d'accéder aux différentes fonctionnalités de l'application.

Le Serveur : Développé en PHP, il reçoit les requêtes du client, traite la logique métier, interagit avec la base de données MySQL si nécessaire, puis renvoie une réponse sous forme de page web ou de données au format approprié (HTML)

Cette architecture assure une communication efficace entre le client et le serveur, et permet de structurer l'application de manière claire et évolutive.

Protocole de communication

L'échange de données entre le navigateur web (client) et le serveur de l'application repose sur la suite de protocoles TCP/IP, base des communications sur Internet. Le protocole IP est responsable de l'adressage unique des machines grâce à l'adresse IP, et de l'acheminement des paquets de données vers leur destination. Le protocole TCP, quant à lui, garantit une transmission fiable et ordonnée des données. Au-dessus de cette couche, l'application utilise le protocole HTTPS (version sécurisée de HTTP) pour assurer des échanges protégés entre le client et le serveur, notamment lors de l'envoi de requêtes ou de la réception de pages web.

3. Conception de la base de données :

3.1 Table candidat

Libellé	type	taille	observation
n_ident_nationale	Varchar	20	Primary Key
nom	Varchar	50	NOT NULL
prenom	Varchar	50	NOT NULL
sexe	ENUM ('H','F')		NOT NULL, H : homme, F:femme
date_naissance	DATE		NOT NULL
commune_naissance	Varchar	50	NOT NULL
wilaya_naissance	ENUM (58 wilaya)		NOT NULL
prenom_pere	Varchar	50	NOT NULL
nom_mere	Varchar	50	NOT NULL
prenom_mere	Varchar	50	NOT NULL
village	Varchar	100	NOT NULL
commune_adresse	Varchar	100	NOT NULL
adresse_wilaya	ENUM (58 wilaya)		NOT NULL
etat_civil	ENUM (M,D,V,C)		C :celibataire,M :marié(e),V :veuf(ve), D :divorcé(e)
n_telephone	Varchar	10	NOT NULL,
nationalite	Varchar	30	NOT NULL
groupe_sanguin	ENUM((A+/B+/AB+/O+)(A-/B-/AB-/O-)		NOT NULL
email	Varchar	60	NOT NULL, index
id_compte	int		NOT NULL**,** Clé étrangère vers comptes (id_compte)
type_permis	ENUM(A1, A2, B, C1 C2, D, E, F)		NOT NULL,

3.2 Table vehicule :

libelle	type	taille	observation
immatriculation	Varchar	20	Primary Key
marque	Varchar	20	NOT NULL
date_assurance	DATE		NOT NULL
type_carburant	ENUM (‘essence’ ,’ diesel ’GPL’, hybride, électrique)		NOT NULL
Boite-vitesse	ENUM (‘automatique’, ‘manuelle’,’semi-automatique’)		NOT NULL

3.3 Table facture :

Libellé	Type	Taille	Description
id_facture	INT	11	NOT NULL, Clé primaire.
n_ident_candidat	VARCHAR	20	NOT NULL, Clé étrangère vers candidat (n_ident_nationale)
description	Varchar		NOT NULL, Description de la facture
montant_total	DECIMAL	10,2	NOT NULL, Montant total dû
date_facture	DATE		NOT NULL

3.4 Table paiement

Libellé	**Type**	Taille	Description
id_paiement	INT	11	Clé primaire.
id_facture	INT	11	NOT NULL, Clé étrangère vers facture(id_facture)
montant_paye	DECIMAL	10,2	NOT NULL, Montant payé
mode_paiement	ENUM (espèce, chèque, virement, etc.)		NOT NULL
date_paiement	DATE		NOT NULL

3.5Table moniteur :

Libellé	Type	Taille	Observation
id_moniteur	int	11	PRIMARY KEY, NOT NULL, AUTO_INCREMENT
n_ident_nationale	Varchar	20	NOT NULL,
nom	Varchar	50	NOT NULL
prenom	Varchar	50	NOT NULL
sexe	ENUM ('M', 'F')		NOT NULL
date_naissance	DATE		NOT NULL
commune_naissance	Varchar	100	NOT NULL
wilaya_naissance	ENUM (58 wilaya)		NOT NULL
village	Varchar	100	NOT NULL
commune_adresse	Varchar	100	NOT NULL
adresse_wilaya	ENUM (58 wilaya)		NOT NULL
etat_civil	ENUM ('C','M','D','V')		NOT NULL
n_telephone	Varchar	10	NOT NULL
nationalite	Varchar	50	NOT NULL, DEFAULT 'Algérienne'
groupe_sanguin	ENUM ('A+','A-','B+','B-','AB+','AB-','O+','O-')		NOT NULL
email	Varchar	100	NOT NULL, INDEX
specialite	ENUM ('A1','A2','B','C1','C2','D','E','F')		NOT NULL
id_compte	int	11	NOT NULL, clé étrangère vers comptes

3.6 Table examen :

Libellé	Type	Taille	Description
id_examen	INT	11	PRIMARY KEY, NOT NULL,
date_examen	DATE		NOT NULL,
heure	TIME		NOT NULL,
type	ENUM (code, conduite, créneau)		NOT NULL,
categorie	ENUM ('A1','A2','B','C1','C2','D','E','F')	2	NOT NULL,
n_ident_candidat	VARCHAR	20	Clé étrangère vers candidat (n_ident_nationale)

3.7 Table seance :

Libellé	Type	Taille	Description
id_seance	INT	11	Primary Key, auto-increment
date_seance	DATE		NOT NULL
heure_debut	TIME		NOT NULL
heure_fin	TIME		NOT NULL
type_seance	ENUM (code, conduite, créneau)		NOT NULL
id_candidat	VARCHAR	20	Clé étrangère vers candidat (n_ident_nationale)
id_moniteur	INT	11	Clé étrangère vers moniteur (id_moniteur)
statut	ENUM (prévu, effectué, annulé)		NOT NULL

3.8Table comptes (moniteurs / candidats/ administrateur) :

Libellé	Type	Taille	Observations
id_compte	INT		PrimaryKey, AUTO_INCREMENT
email	VARCHAR	60	NOT NULL, UNIQUE(pour admin)
mot_de_passe	VARCHAR	255	NOT NULL
role	ENUM(‘administrateur' moniteur', 'candidat')		NOT NULL

3.9 Resultat_Examen :

Libellé	Type	Taille	Description
id_resultat	INT	11	Primary Key
id_examen	INT	11	Clé étrangère vers examen (id_examen)
resultat	ENUM (réussi ou échoué)		NOT NULL

Modèle Logique de Données (MLD) :

Le modèle logique des données représente la structure des données sans dépendance à un langage de programmation. Il formalise les entités, leurs attributs et leurs relations :

. Candidat :(n_ident_nationale, nom ,prenom , sexe , date_naissance , commune_naissance, wilaya_naissance , prenom_pere , nom_mere , prenom_mere , village , commune_adresse , adresse_wilaya , etat_civil , n_telephone , nationalité , groupe_sanguin , email , # id_compte, type _permis)

. Vehicule : (immatriculation , marque , date_assurance, type_carburant , Boite_vitesse)

. Facture : (id_facture , # n_ident_candidat , description , montant_total , date_facture)

. Paiement : (id_paiement , # id_ facture , montant_paye , mode_paiement , date_paiement)

. Moniteur : (id_moniteur, n_ident_national , nom , prenom , sexe , date_naissance ‘ commune_naissance , wilaya_naissance , village , commune_adresse , adresse_wilaya , etat_civil , n_telephone , nationalité , groupe_sanguin , email , specialité , # id_compte)

. Examen : (id_examen , date_examen , heure , type, categorie , # n_ident_candidat)

. Seance : (id_seance , date_seance , heure_debut , heure_fin , type_seance , # id_candidat , # id_moniteur , statut)

. Compte : (id_compte , email , mot_de_passe , role)

. Resultat_examen : (id_resultat , # id_ examen , resultat)

4. Maquette de l’interface utilisateur (authentification) :

4.1 Création de compte :

T01 : accès à l’application : L’utilisateur accède à l’application via son navigateur

T02 : ouverture de la page de connexion : L’application affiche la page de connexion avec options :

- Connexion
- Créer un compte

TA: cliquer sur bouton « créer un nouveau compte »

TA1 : saisir un email

TA2 : ressaisir l’email

TA3 : saisir Le mot de passe : L’utilisateur entre un mot de passe conforme aux règles de sécurité :

- Minimum 8 caractères
- Majuscules, minuscules, chiffres, symboles

TA4 : ressaisir le mot de passe

TA5 : tapez sur le bouton « créer » : Envoie les données au serveur.

(les comptes des candidats et moniteurs sont créés par l’administrateur)

Gestion de l’authentification et réinitialisation du mot de passe

Si Connexion

TB : Cliquer sur le bouton <se connecter>

TB1 : L’utilisateur saisit son email.

Si mot de passe oublié :

TB11 : Cliquer sur le bouton< mot de passe oublié>

TB112: saisir l’email

TB113: L'utilisateur clique sur le bouton « Se connecter ».

TB114 : Cliquer sur le bouton<envoyer>

TB115 :modifier le mot de passe

Sinon :

TB12 :saisir Le mot de passe

TB121 : Cliquer sur le bouton<se connecter>

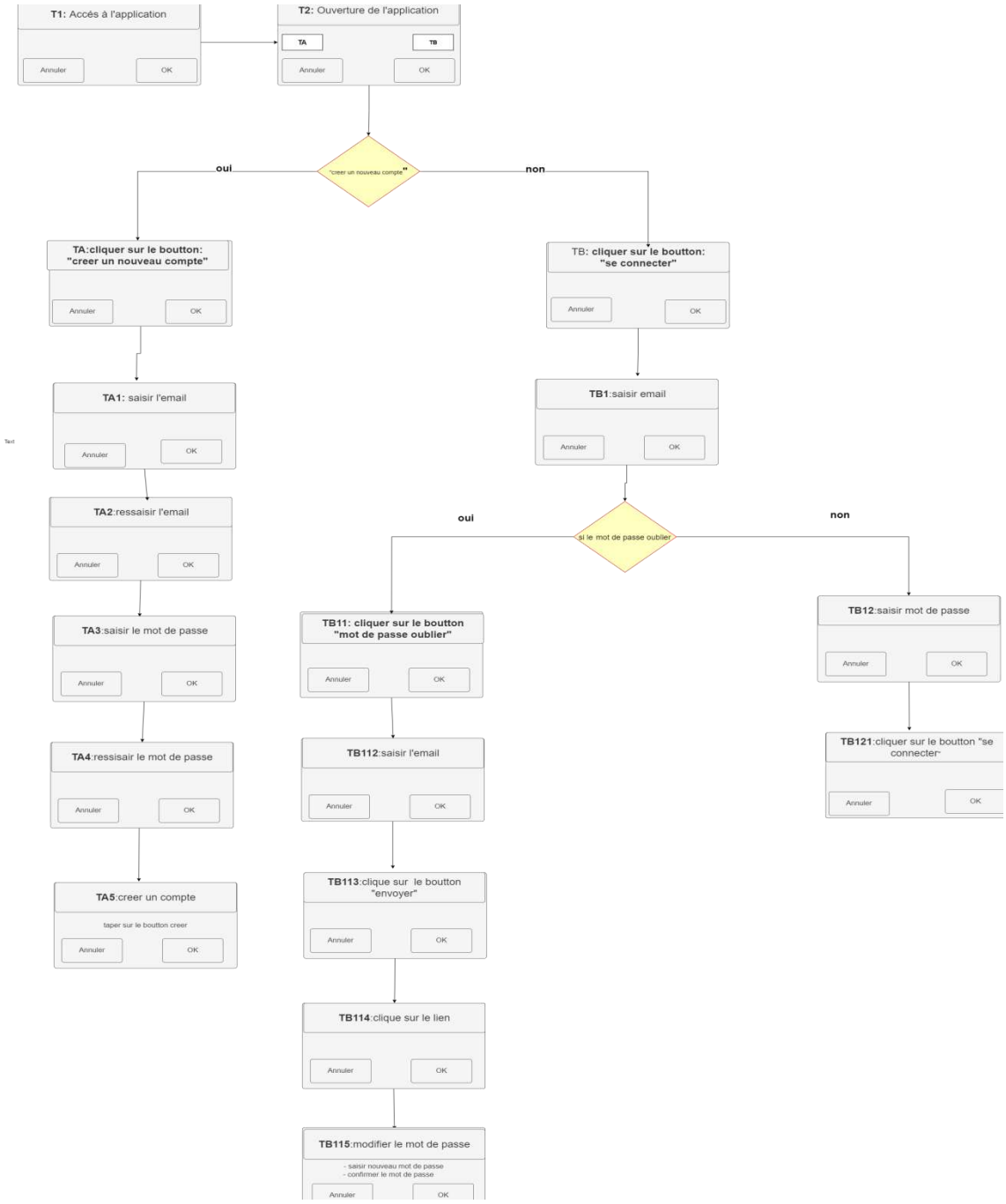


Figure 18 : «Maquette -authentification- »

Conclusion : Vers une Base Technique Solide et Maîtrisée

Ce chapitre a jeté les fondations techniques essentielles pour la réalisation de l'application en mettant en lumière les outils et langages les plus adaptés aux exigences du projet. Les choix technologiques ont été effectués avec soin afin de garantir la compatibilité, la performance et la fiabilité de la solution finale. L'adoption d'une architecture web structurée assure une organisation claire entre les différentes composantes du système, facilitant ainsi son évolution et sa maintenance. La conception rigoureuse de la base de données permet une gestion efficace et sécurisée des informations, en adéquation avec les besoins fonctionnels identifiés. De plus, les premières maquettes d'interface posent les bases d'une expérience utilisateur fluide et intuitive. Cette préparation technique offre un cadre stable et cohérent pour entamer sereinement la phase de développement de l'application.

CHAPITRE 4 :

DEVELOPPEMENT DE LA SOLUTION

Après avoir défini les besoins, modélisé le système et établi les choix technologiques, ce chapitre marque le passage concret à la réalisation de l'application de gestion pour auto-école. Il décrit de manière détaillée les étapes de développement mises en œuvre, depuis l'installation de l'environnement de travail jusqu'à la création des différentes interfaces et fonctionnalités. L'objectif est de transformer la conception théorique en une solution fonctionnelle, stable et adaptée aux attentes des utilisateurs. Ce processus de développement s'appuie sur les spécifications établies précédemment, en veillant à respecter les principes de clarté, de maintenabilité et de sécurité.

1. Structure du projet :

Avant de plonger dans les aspects fonctionnels et visuels de l'application, il est essentiel de présenter l'organisation générale du projet. Cette structure reflète la répartition logique des fichiers et dossiers qui composent l'application web de gestion d'auto-école. Elle facilite la lisibilité du code, la maintenance du projet ainsi que la séparation des rôles (administrateur, moniteur,candidat).
L'arborescence ci-dessous montre les différents répertoires utilisés, chacun ayant un rôle bien défini dans le fonctionnement global de l'application.

- Arborescence des dossiers :

autoecole :

1.1 Config/ :

db.php

1.2 auth/ :

créer_admin.php

login_admin.php

login_candidat.php

login_moniteur.php

logout.php

1.3 index.php
1.4 admin/ :

admin_home.php

ajouter_candidat .php

ajouter_facture.php

ajouter_moniteur.php

ajouter_paiement.php

ajouter_vehicule.php

liste_paiements.php

modifier_candidat.php

modifier_moniteur.php

modifier_seance.php

modifier_vehicule.php

planning_examen.php

planning_seance.php

recherche_candidat.php

recherche_examen.php

recherche_moniteur.php

recherche_vehicule.php

resultat_examen.php

supprimer_candidat.php

supprimer_examen.php

supprimer_moniteur.php

supprimer_seance.php

supprimer_vehicule.php

1.5 assets/ :

1.5.1 :css / :

style.css

style2.css

style3.css

style4.css

style5.css

style6.css

style7.css

style8.css

style9.css

style10.css

style11.css

style12.css

style13.css

1.5.2 images/ :

Code.jpg

Voiture.jpeg

1.5.3 js/

script.js

1.6 candidat /:

apprendre.php

candidat_home.php

examens_candidat.php

fiche_candidat.php

resultats_examens.php

seances_candidat.php

1.7 moniteur / :

candidat.php

fiche_moniteur.php

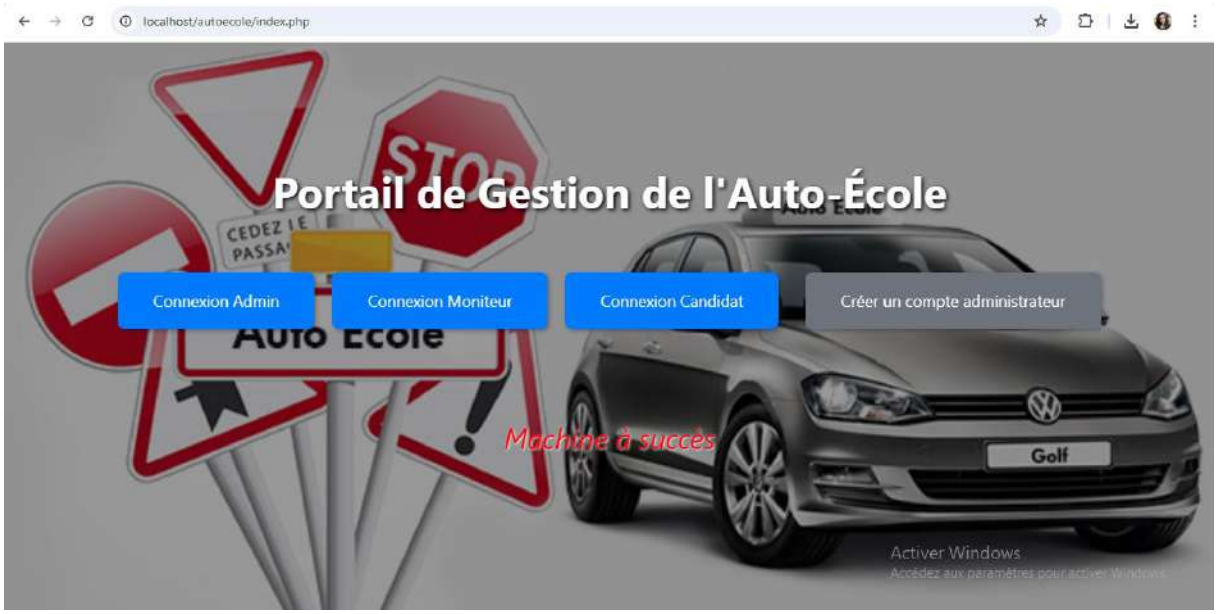
moniteur_home.php

planning_seances.php

2 : Présentation de l'Interface Utilisateur et Fonctionnalités de l'Application :

2.1 Interface générale :

Cette figure représente la page d'accueil principale de l'application de gestion d'auto-école. Il vérifie d'abord s'il existe déjà un compte administrateur dans la base de données, puis affiche les boutons de connexion selon le rôle (administrateur, moniteur, candidat). Si aucun compte administrateur n'existe, un lien permettant d'en créer un est proposé.



2.1.1 créer un compte administrateur :

Cette figure est pour la création du tout premier compte administrateur avec des vérifications strictes sur l'email et le mot de passe. Il s'assure qu'un seul compte admin peut exister et sécurise les données avec un hachage du mot de passe avant enregistrement. Une fois le compte est créé , le bouton créer un compte administrateur va se disparaitre .



3 . Administrateur :

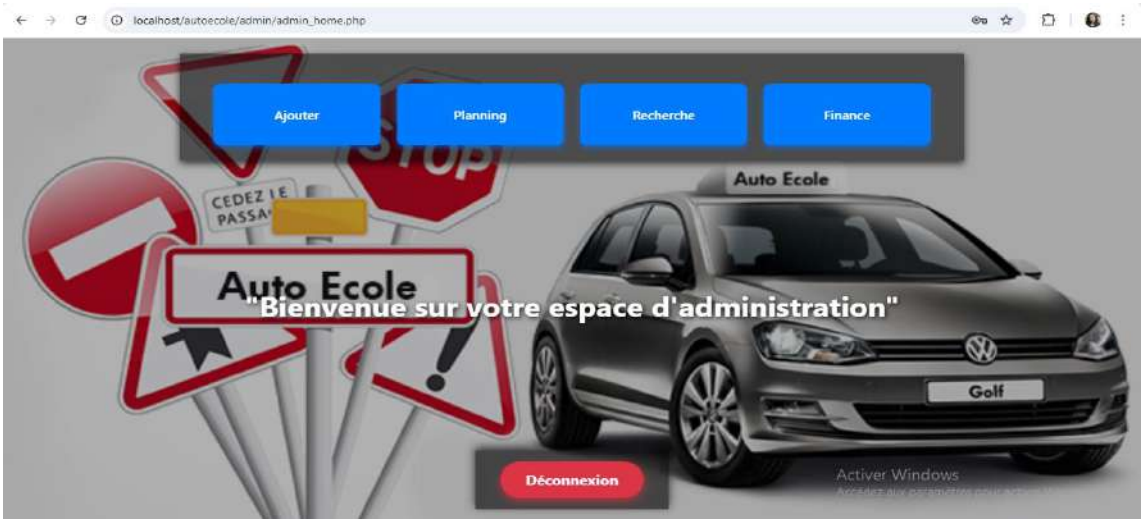
3.1 Connexion admin :

Cette figure permet à l'administrateur de se connecter de manière sécurisée. Il vérifie l'existence du compte via l'email, compare le mot de passe saisi avec le mot de passe haché en base, puis ouvre une session et redirige vers l'espace admin en cas de succès.

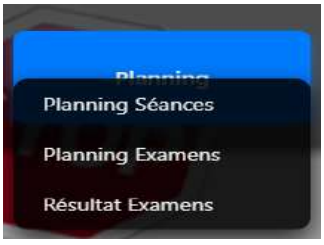


3.2 L'espace administrateur :

Cette interface sécurisée est accessible uniquement au administrateurs connectés. Elle propose un tableau de bord avec des menus dynamiques pour gérer les candidats, moniteurs, véhicules, plannings, examens et finances. Un fond personnalisé avec une image de voiture améliore l'esthétique de la page





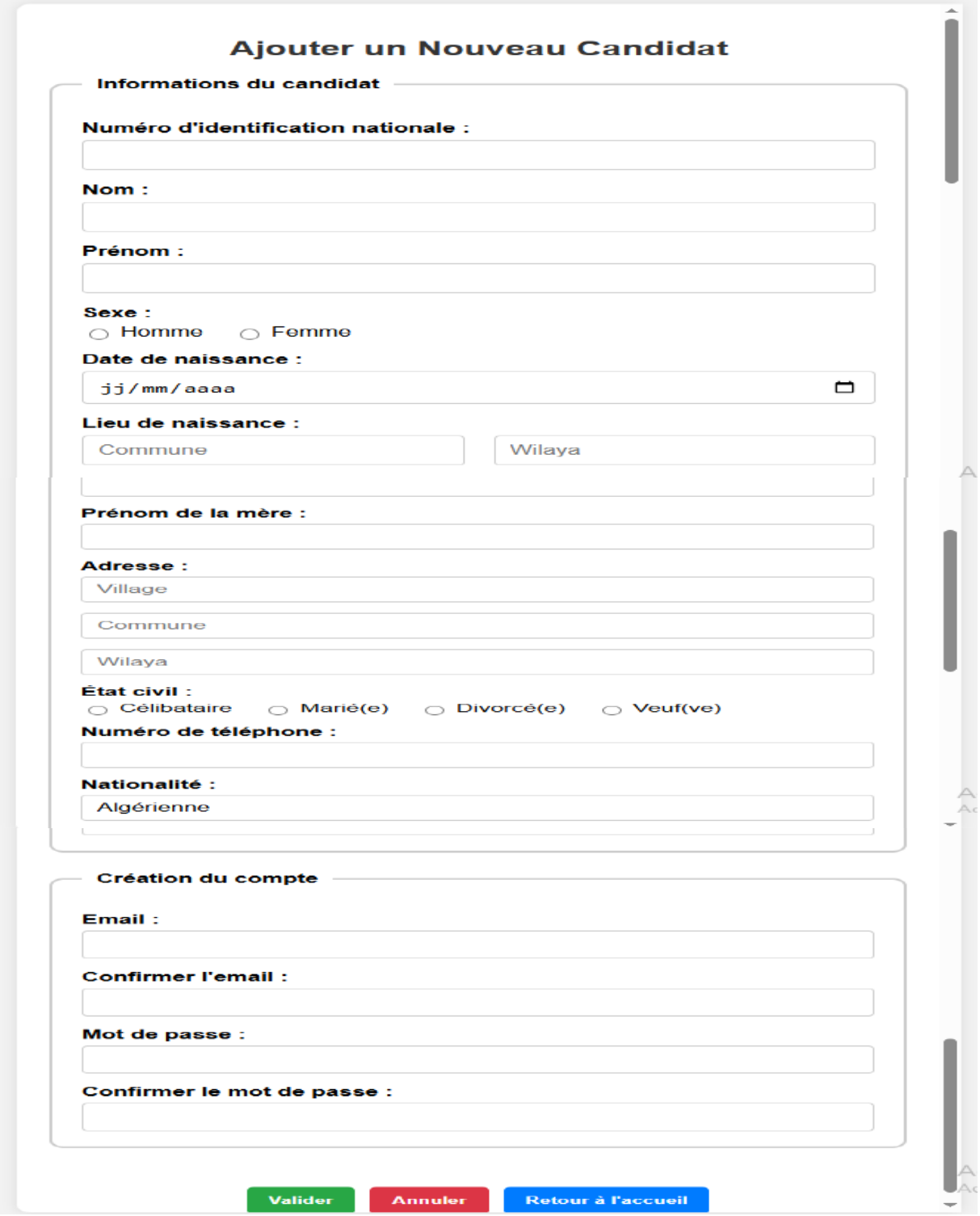




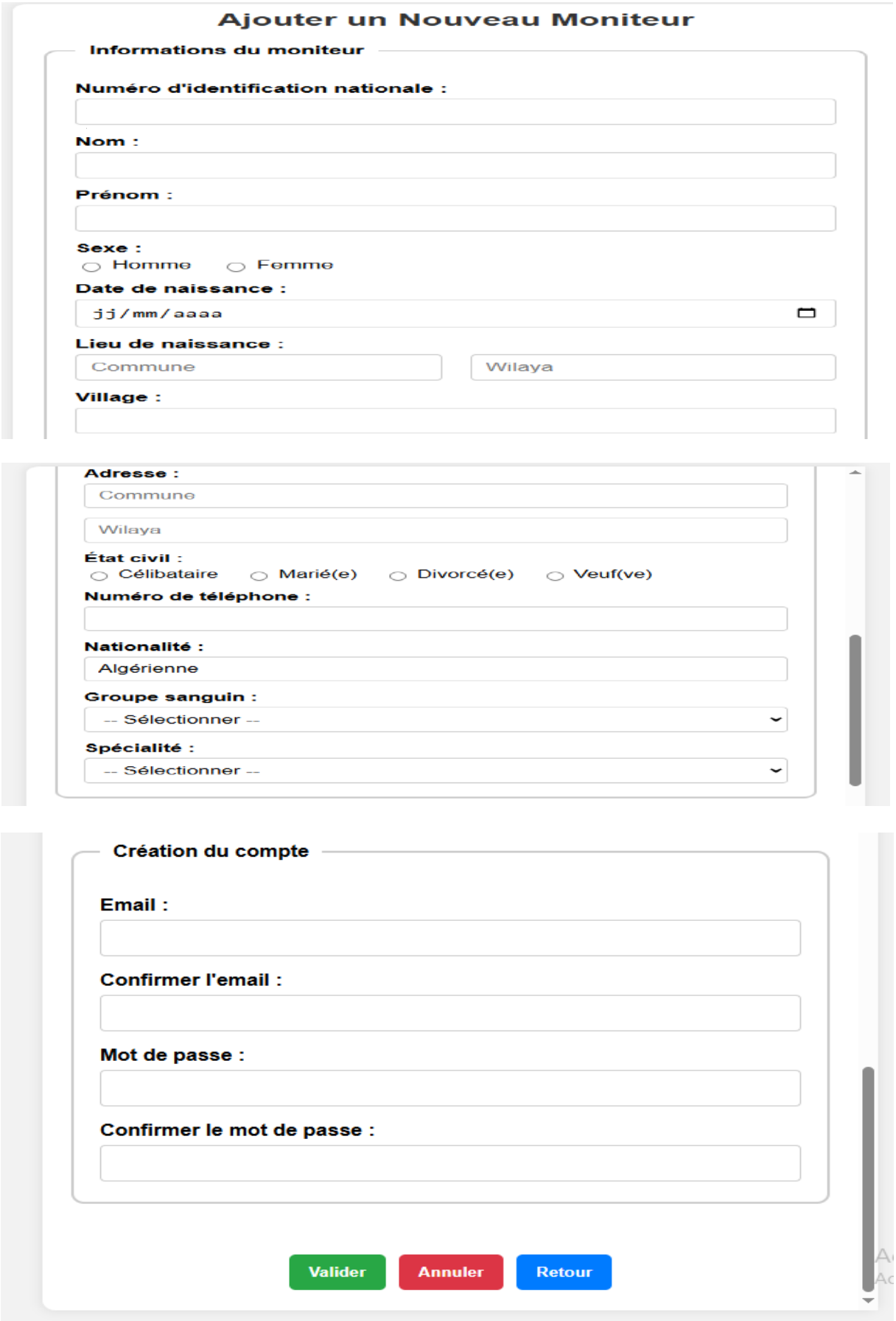


3.3 Ajouter candidat :

Conformément à la loi n° 18-07 du 10 juin 2018 relative à la protection des personnes physiques dans le traitement des données à caractère personnel en Algérie, les informations collectées sont nécessaires à l'enregistrement du candidat et à la création de son compte. Ces données sont traitées de manière confidentielle et utilisées exclusivement à des fins de gestion administrative de l'auto-école



3.4Ajouter moniteur :



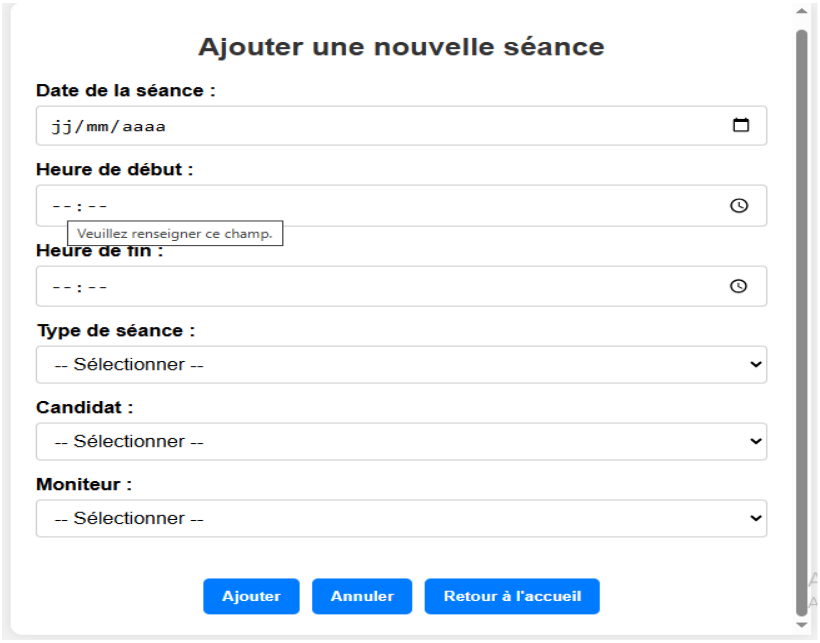
3.5Ajouter vehicule :

Ce formulaire permet l'ajout d'un véhicule en vérifiant les champs saisis par l'utilisateur. En cas de validation réussie, les données sont enregistrées dans la base via une requête préparée. Des messages indiquent si l'opération a réussi ou s'il y a des erreurs. »



3.6 Planning séances :

Cette interface permet d'enregistrer une nouvelle séance dans le système en récupérant les informations saisies par l'utilisateur, telles que le moniteur et le candidat. Une vérification des champs est effectuée avant l'insertion sécurisée des données dans la base via une requête préparée. Des messages clairs sont ensuite affichés pour informer de la réussite ou des éventuelles erreurs. »



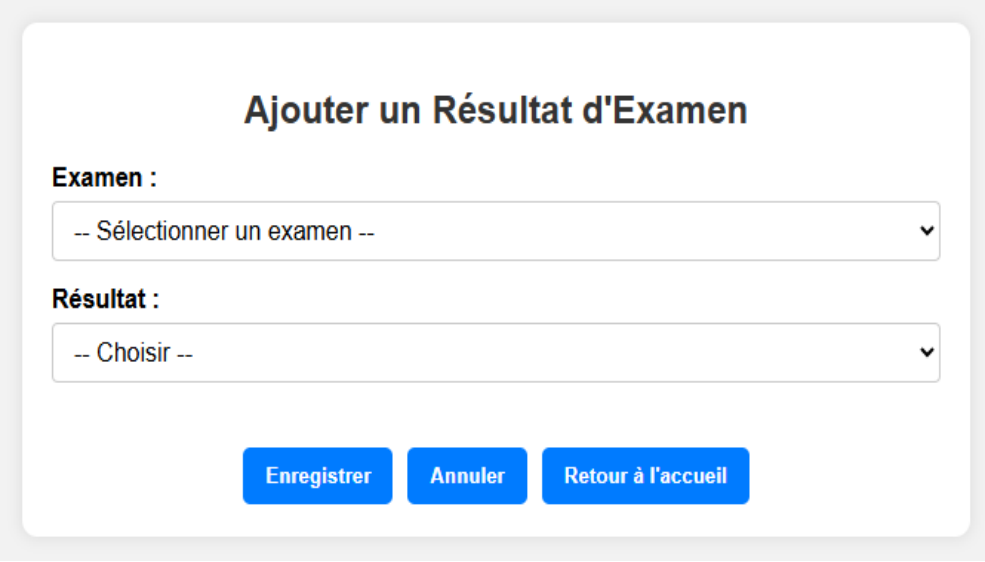
3.7 Planning examens :

Cette interface permet d'enregistrer les examens à venir, en validant les informations fournies avant de les stocker en base de données. L'utilisateur est informé du succès ou des erreurs rencontrées



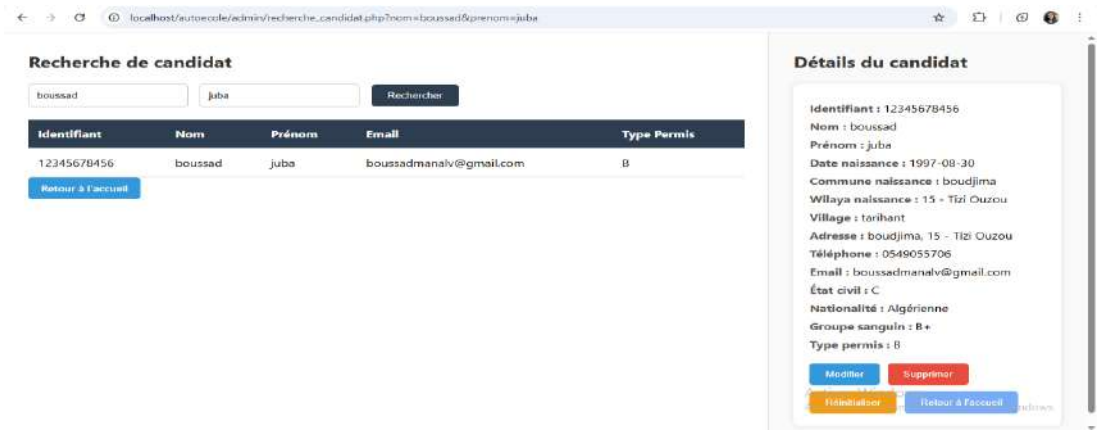
3.8 Resultat examens :

Cette interface permet d'ajouter les résultats des examens en s'assurant de la validité des données. Les résultats sont ensuite enregistrés et un retour est affiché à l'utilisateur.

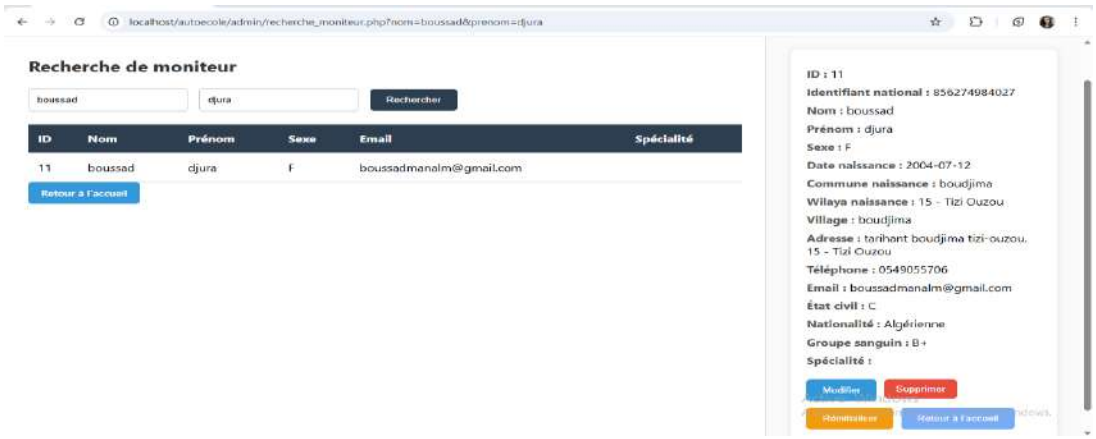


3.9Recherche candidat :

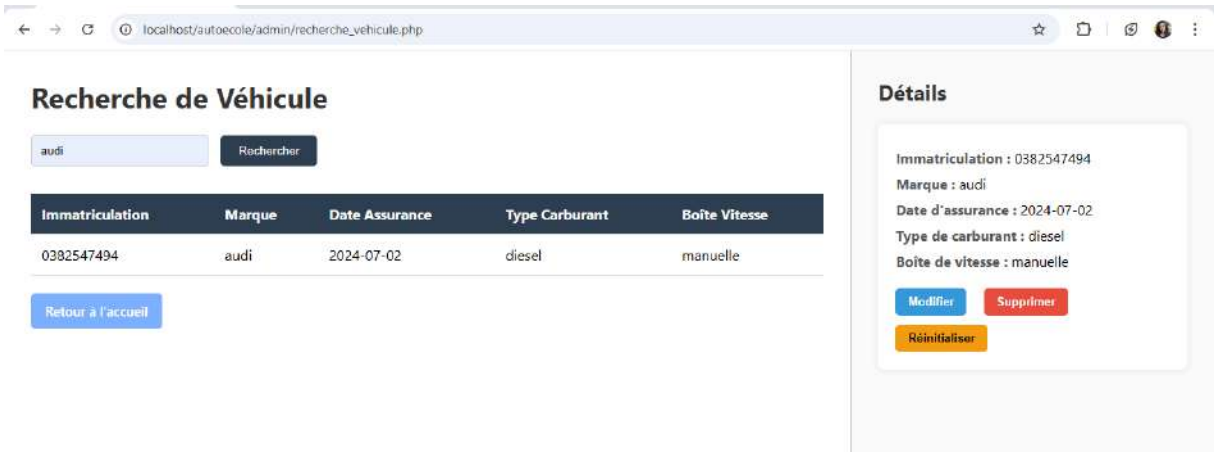
Cette interface permet de rechercher des candidats dans la base de données en fonction des critères saisis par l'utilisateur, (nom et prénom). Il effectue une requête sécurisée et affiche une liste complète des candidats correspondants avec leurs informations en tableau , in clique sur le candidat souhaité et on voit à droite sa fiche détaillé . Des options pour **modifier ou supprimer** un candidat sont proposées directement dans les résultats.



3.10 Recherche moniteur :



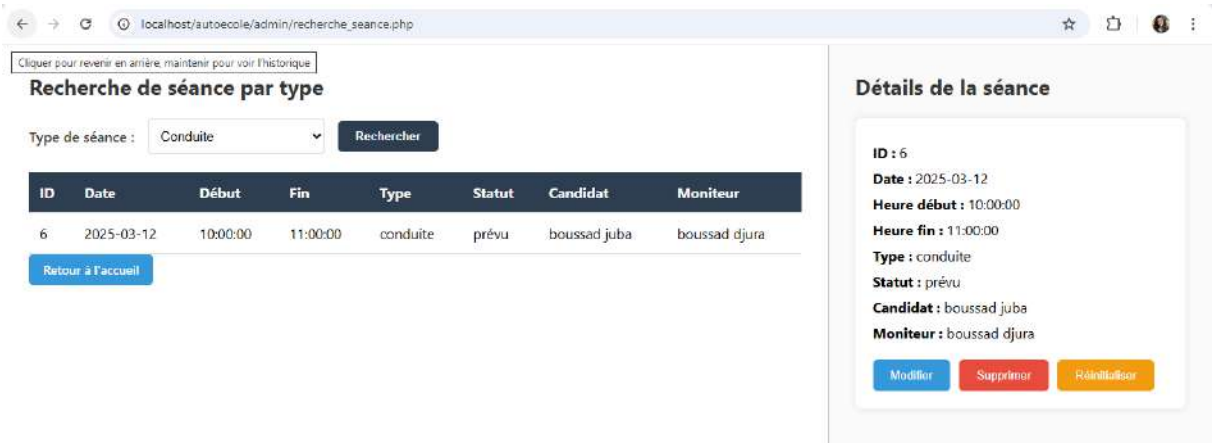
3.11 Recherche vehicule :



3.12 Recherche examen :



3.12 Recherche séance :



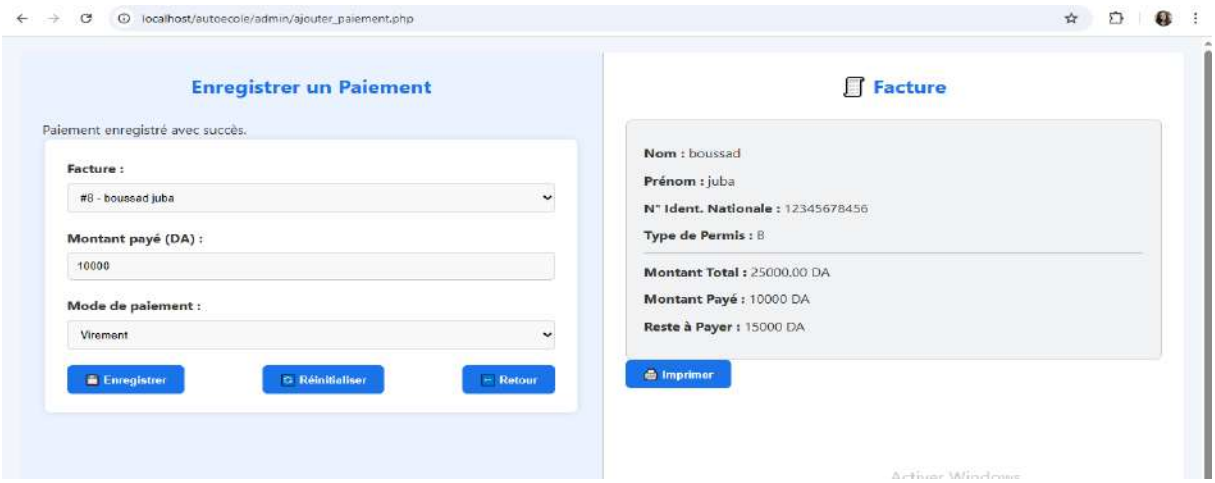
3.13 Créer facture :

Cette interface permet à l'administrateur de générer une nouvelle facture pour un candidat en sélectionnant celui-ci, en saisissant une description et le montant total dû. Le système vérifie les champs obligatoires avant d'insérer les données dans la base de manière sécurisée. En cas de succès ou d'erreur, un message clair est affiché à l'utilisateur.



3.14 enregistrer paiement :

Cette interface permet à un administrateur d'enregistrer un paiement associé à une facture en récupérant les données saisies, les validant puis en les insérant dans la base de données. Il affiche ensuite un récapitulatif du paiement avec les détails du candidat et du montant restant à régler.



3.15 enregistrer paiement :

Cette interface affiche la liste complète des paiements enregistrés dans le système, en précisant les informations liées à chaque paiement comme le montant, le mode de paiement et la facture associée. Il permet à l'administrateur de consulter rapidement l'historique financier des candidats. L'affichage est structuré et facilite la vérification des transactions effectuées



Des messages d'erreurs et de succès : (plusieurs)





4. Moniteur :

4.1 Connexion moniteur :

Le module de connexion moniteur permet à un moniteur d'accéder à son espace personnel via un compte préalablement créé par l'administrateur. L'authentification est sécurisée par un identifiant et un mot de passe, et seules les informations enregistrées par l'administrateur permettent l'accès



4.2 espace moniteur :

L'espace moniteur offre une interface simplifiée et intuitive permettant aux moniteurs de consulter efficacement leurs activités. Il se compose de trois sections principales accessibles via des boutons :

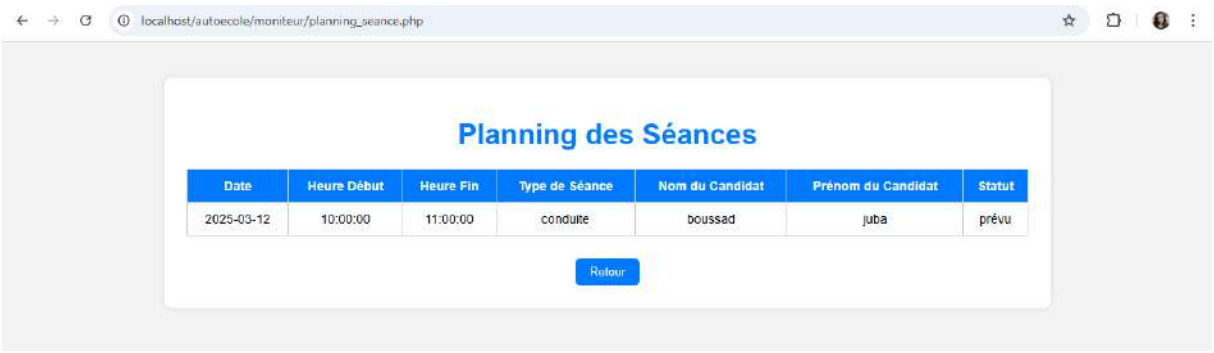


4.3 fiche personnelle : (celle du moniteur connecté)



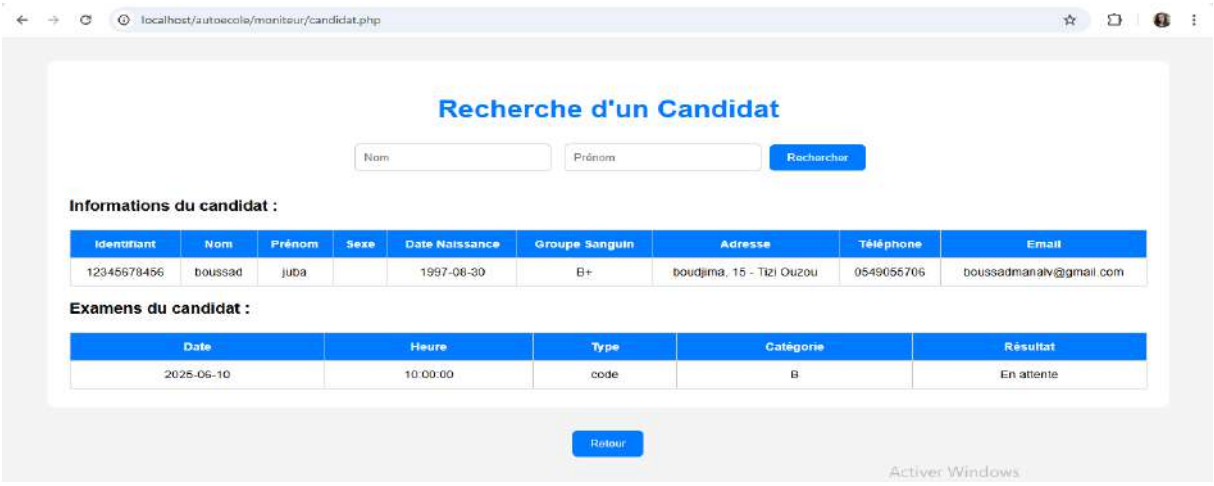
4.4 planning des seances :

Le planning des séances est automatiquement généré pour le moniteur connecté dès qu'il accède à son espace. Le système récupère depuis la base de données toutes les séances qui lui sont attribuées



4.5 candidat :

Le moniteur peut rechercher un candidat par nom et prénom. Il accède alors à ses informations personnelles, aux détails de son examen, ainsi qu'à son résultat. Cela permet un suivi rapide et efficace.



5. candidat :

5.1 Connexion candidat : (créé par l'administrateur)



5.2 espace candidat :

L'espace candidat offre une interface simplifiée et intuitive permettant aux candidats de consulter efficacement leurs activités. Il se compose de trois sections principales accessibles via des boutons :



5.3 fiche personnelle : (celle du candidat connecté)



Fiche Personnelle du Candidat

Identifiant national : 12345678456
Nom : boussad
Prénom : juba
Date de naissance : 1997-08-30
Commune de naissance : boudjima
Wilaya de naissance : 15 - Tizi Ouzou
Prénom du père : hamid
Nom de la mère : tagzirt
Prénom de la mère : samia
Village : tarihant
Commune adresse : boudjima
Adresse wilaya : 15 - Tizi Ouzou
État civil : C
Numéro de téléphone : 0549055706
Nationalité : Algérienne
Groupe sanguin : B+
Email : boussadmanalv@gmail.com
Type de permis : B

Retour

5.4 Séances :

Le planning des séances est automatiquement généré pour le candidat connecté dès qu'il accède à son espace. Le système récupère depuis la base de données toutes ces séances



5.5 Examens :

Le planning des examens est automatiquement généré pour le candidat connecté dès qu'il accède à son espace. Le système récupère depuis la base de données ses examens .



5.5Résultat d'examens :

Le résultat est automatiquement généré pour le candidat connecté dès qu'il accède à son espace. (Réussi, echoué, en attente)



5.6 Apprendre : Ou le candidat trouve le code de la route pour s'entrainer



Conclusion :

Ce chapitre a présenté en détail les étapes concrètes du développement de l'application de gestion d'auto-école, depuis la structuration du projet jusqu'à la mise en œuvre des différentes interfaces et fonctionnalités. Chaque rôle (administrateur, moniteur, candidat) dispose d'un espace personnalisé et sécurisé, répondant à ses besoins spécifiques. L'approche adoptée a permis de transformer les spécifications initiales en une solution fonctionnelle, intuitive et conforme aux attentes des utilisateurs. Grâce à une organisation claire, un codage structuré et une attention portée à la validation des données, l'application garantit à la fois performance, sécurité et facilité d'utilisation.

Conclusion et perspectives :

1. Bilan du projet :

Le développement de l'application web de gestion pour auto-école a permis de répondre efficacement aux besoins identifiés en amont du projet. Grâce à une architecture bien définie, une interface utilisateur claire et un système d'authentification par rôle (administrateur, moniteur, candidat), l'application facilite la gestion quotidienne d'une auto-école, tout en assurant la sécurité et la traçabilité des informations.

Le projet a couvert l'ensemble des fonctionnalités essentielles : gestion des comptes utilisateurs, planification des séances et examens, suivi des paiements, visualisation des résultats, ainsi que l'accès à des outils pédagogiques pour les candidats. L'intégration d'une base de données structurée et de requêtes sécurisées a permis de garantir la fiabilité du système.

Sur le plan technique, le projet a renforcé les compétences en développement web (PHP, HTML/CSS, JavaScript) et en gestion de bases de données relationnelles. Il a également mis en évidence l'importance de la modélisation, de la validation des entrées, de la répartition des rôles et de l'ergonomie dans le cycle de développement logiciel.

2. Difficultés rencontrés :

-Conception des interfaces : Trouver un bon équilibre entre simplicité d'utilisation et fonctionnalités complètes a été un vrai défi.

-Temps limité

Le temps pour faire tout le projet était court, il a fallu bien s'organiser pour finir à temps tout en gardant une bonne qualité.

- Relations en base de données

Relier correctement les tables (candidat, moniteur, séance, examen, paiement) a été difficile pour éviter les erreurs et doublons.

3. Amélioration future possible :

- **Paiement en ligne** : Intégrer un module de paiement sécurisé pour régler les factures directement via l'application.

- **Support mobile** : Adapter l'application pour une utilisation sur Smartphone (responsive ou application mobile dédiée).

- **Ajout de fichiers de formation** : Intégrer des vidéos ou des quiz interactifs dans la section "Apprendre" pour mieux préparer les candidats.

- **Mot de passe oublié** : Ajouter une fonctionnalité permettant aux utilisateurs de réinitialiser leur mot de passe en cas d'oubli, par e-mail ou question secrète.

Bibliographie :

Supports pédagogiques :

Mémoire : nous nous sommes inspirés des années précédentes de notre université

Ressources en ligne :

PHP tutoriels : PHP Full Course for non-haters
HTML, CSS, JavaScript Tutoriaux. **https://www.w3schools.com/**

Normes et réglementations

République Algérienne Démocratique et Populaire. Loi n°18-07 du 10 juin 2018 relative à la protection des données personnelles.

Outils et technologies utilisés

XAMPP. *Serveur local Apache + MySQL + PHP*.
https://www.apachefriends.org/fr/index.html

Visual Studio Code. *Éditeur de code*. https://code.visualstudio.com/